

AI-Based Food Recognition with Nutrition-Aware Recommendations

A Report submitted
in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Jaspreet Singh Enrollment No.: 2022BCSE019

Inderpal Singh Enrollment No.: 2022BCSE037

Under the guidance of

Dr. Lavanya Madhuri Bollipo

Supervisor

Dr. Insha Majeed

Co-Supervisor

Department of Computer Science and Engineering
National Institute of Technology Srinagar
Kashmir 190006, INDIA

June 2026

CERTIFICATE

This is to certify that the project report entitled **AI-Based Food Recognition with Nutrition-Aware Recommendations** submitted by **Jaspreet Singh (2022BCSE019)** and **Inderpal Singh (2022BCSE037)** to the Department of Computer Science and Engineering, National Institute of Technology Srinagar, in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a bona fide record of project work carried out under the supervision of Dr. Lavanya Madhuri Bollipo and co-supervision of Dr. Insha Majeed.

The work presented in this report has been prepared as a final year project in the area of computer vision, full-stack web engineering, nutrition data integration, and recommendation systems. To the best of our knowledge, the work described here has not been submitted elsewhere for the award of any degree or diploma.

Dr. Lavanya Madhuri Bollipo

Supervisor

Department of Computer Science and
Engineering

National Institute of Technology Srinagar

Dr. Insha Majeed

Co-Supervisor

Department of Computer Science and
Engineering

National Institute of Technology Srinagar

STUDENT DECLARATION

We declare that this project report titled **AI-Based Food Recognition with Nutrition-Aware Recommendations** is submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering. The project is a record of original work carried out by us under the supervision of Dr. Lavanya Madhuri Bollipo and co-supervision of Dr. Insha Majeed.

The matter embodied in this project, in full or in part, has not been submitted to any other institution or university for the award of any degree or diploma. We also declare that the work submitted by us is original and that all external datasets, software libraries, documentation sources, and research references used in the project have been acknowledged appropriately.

Jaspreet Singh

2022BCSE019

Inderpal Singh

2022BCSE037

Srinagar, 190006

June 2026

ACKNOWLEDGEMENTS

We express our sincere gratitude to our project guide, Dr. Lavanya Madhuri Bollipo, and co-supervisor, Dr. Insha Majeed, for their guidance, review, and technical direction throughout this final year project. Their feedback was especially useful when the work moved from a broad idea into practical problems such as cleaning dataset labels, checking food-class images, correcting nutrition mappings, and connecting the model output to a usable web interface.

We are thankful to the Department of Computer Science and Engineering, National Institute of Technology Srinagar, for providing the academic environment and opportunity to carry out this work. We also acknowledge the open-source communities behind Python, FastAPI, Next.js, Ultralytics YOLO, PyTorch, and the scientific computing tools used in this project. The availability of these tools allowed us to spend more time on project-specific issues such as class normalization, nutrition validation, and interface behavior.

We also thank our families, friends, and peers for their support, feedback, and patience during the design, implementation, debugging, testing, and documentation phases of the work.

Jaspreet Singh
2022BCSE019

Inderpal Singh
2022BCSE037

ABSTRACT

Manual food logging becomes difficult mainly because it repeats the same small tasks after every meal. The user has to identify each dish, estimate the serving, find a matching nutrition entry, and repeat the process for the remaining items on the plate. Indian meals make this task less direct because one plate may contain rice, dal, chutney, curry, fried items, sweets, and beverages, often with regional names or household variations. This project was developed to reduce that effort by building an image-based Indian food recognition system with nutrition-aware recommendations. The system takes a food image as input, detects visible food items through a YOLO-based object detector, links the detected labels to Indian nutrition records, estimates calories and macronutrients per 100 grams, and presents dietary suggestions through a web application.

The implemented prototype consists of a FastAPI backend, a Next.js frontend, a SQLite nutrition database built mainly from the Indian Nutrient Database, and a merged YOLO-format dataset with 72 canonical Indian food classes. Five source datasets were combined, but this was not a direct file merge. The class names had to be cleaned because spelling differences, regional words, and duplicate labels could create separate detector outputs for the same dish. Examples such as idly and idli, or satham and rice, were handled before training. The final balanced export contains 48,693 image-label pairs across training, validation, and test splits. Visual verification sheets were also prepared from actual dataset images. These sheets confirmed that all 72 classes had usable samples and also pointed out a few annotation-level issues that should be corrected in later iterations.

The final detector was trained using YOLO11s on the 72-class dataset. The best model achieved 0.830 mAP@50 and 0.692 mAP@50:95 on validation, and 0.820 mAP@50 and 0.680 mAP@50:95 on the held-out test split. The nutrition layer maps all 72 dataset classes to database entries through explicit semantic mappings and verified supplemental rows before fuzzy matching is used. This was added after early tests showed that name similarity alone could return a nutritionally wrong food. The backend includes endpoints for image analysis, nutrition validation, macro calculation, daily

goals, health conditions, and recommendation generation. The frontend supports image upload, detection visualization, serving adjustment, macro tracking, and personalized guidance. The report documents the design decisions, implementation work, validation results, limitations, and future improvements of the system.

Contents

CERTIFICATE	i
STUDENT DECLARATION	iii
ACKNOWLEDGEMENTS	v
ABSTRACT	vii
LIST OF FIGURES	xiii
LIST OF TABLES	xv
ABBREVIATIONS AND NOTATIONS	xvii
1 INTRODUCTION	1
1.1 Overview	1
1.2 Motivation	2
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Scope of the Project	3
1.6 Major Contributions	4
1.7 Report Organization	5
2 LITERATURE REVIEW	7
2.1 Scope of the Review	7
2.2 Expanded Paper Summaries	7
2.2.1 Vision and Object Detection	7
2.2.1.1 IndianFoodNet	7
2.2.1.2 Khana Dataset	8
2.2.1.3 Enhanced Food Image Recognition	8

2.2.1.4	Single-item Training for Multi-dish Recognition . . .	8
2.2.1.5	Lightweight and Parameter-Optimized CNN	9
2.2.1.6	What is in an Indian Thali?	9
2.2.2	Nutrition Databases and Semantic Mapping	9
2.2.2.1	Development of INDB	9
2.2.2.2	Indian Food Composition Tables	10
2.2.2.3	Synergistic Frameworks for Indian Food Computing .	10
2.2.2.4	Nutrition Information Estimation from Food Photos .	10
2.2.2.5	Label AI	10
2.2.3	Recommendations, LLMs, and System Integration	11
2.2.3.1	Diet Engine	11
2.2.3.2	AI-driven Personalized Meal Planning	11
2.2.3.3	NutriGen	11
2.2.3.4	LLMs and Computer Vision for Personalized Nutri- tion Advice	12
2.2.3.5	AI-Based Nutrition Recommendation System	12
2.2.3.6	Evaluation of ChatGPT for Nutrient Estimation . . .	12
2.2.3.7	Visual Nutrition Analysis	12
2.3	Comparative Summary	13
2.4	Research Gap	14
3	METHODOLOGY	15
3.1	Methodological Overview	15
3.2	YOLO Detection Approach	16
3.3	Source Dataset Collection	17
3.4	Canonical Class Normalization	17
3.5	YOLO Label Representation	18
3.6	Dataset Splitting and Augmentation	18
3.7	Nutrition Database Methodology	19
3.8	Nutrition Mapping Methodology	20
3.9	Macro Calculation Methodology	20
3.10	Validation Methodology	21
4	SYSTEM DESIGN	23
4.1	Design Goals	23
4.2	High-Level Architecture	23

4.3	Runtime Data Flow	24
4.4	Backend Design	25
4.5	Frontend Design	26
4.6	Database Design	26
4.7	API Design	27
4.8	Security and Validation Design	28
4.9	Design Trade-Offs	29
5	IMPLEMENTATION	31
5.1	Development Environment	31
5.2	Dataset Build Script	31
5.3	Final YOLO Training Notebook	32
5.4	Visual Class Verification Implementation	32
5.5	Nutrition Database Build Implementation	33
5.6	Vision Service Implementation	33
5.7	Nutrition Service Implementation	34
5.8	Recommendation and Macro Implementation	34
5.9	Frontend Implementation	34
5.10	Code Organization	35
5.11	Implementation Challenges	36
6	EXPERIMENTS AND VALIDATION	37
6.1	Experimental Scope	37
6.2	Dataset Validation	38
6.3	YOLO Training and Detector Evaluation	38
6.4	Class Verification Experiment	43
6.5	Nutrition Database Validation	43
6.6	Nutrition Mapping Coverage Experiment	44
6.7	Service Lookup Validation	45
6.8	Backend API Validation	45
6.9	Frontend Workflow Validation	46
6.10	Macro Calculation Validation	46
6.11	Experimental Findings	47
7	RESULTS	49
7.1	Dataset Engineering Results	49
7.2	Detector Training Results	50

7.3	Nutrition Mapping Results	50
7.4	Application Results	51
7.5	Macro and Recommendation Results	51
7.6	Result Synthesis	52
8	DISCUSSION	53
8.1	Interpretation of Results	53
8.2	Strengths of the System	53
8.3	Limitations	54
8.4	Ethical and Practical Considerations	55
8.5	Why Supplemental Rows Were Added	55
8.6	Lessons Learned	56
9	CONCLUSION	57
10	FUTURE WORK	59
10.1	Self-Trained Recommendation Model	59
10.2	Portion Size Estimation	59
10.3	Mobile Deployment	60
10.4	Personalized Long-Term Tracking	60
10.5	Clinical Safety and Explainability	60
A	DATASET CLASSES AND NUTRITION MAPPINGS	65
A.1	Expanded Dataset Class List	65
A.2	Class-to-INDB Mapping Table	65
A.3	Known Annotation Issues	68
A.4	Visual Verification Sheet Set	69
A.5	Per-Class Dataset Counts	84
B	API RESPONSE EXAMPLES	89
B.1	Analyze Food Response	89
B.2	Macro Calculation Request	90
B.3	Macro Calculation Response	90
B.4	No Food Response	91
B.5	Macro Target Reference Tables	91

List of Figures

1.1	End-to-end workflow of the implemented food recognition and nutrition system	2
3.1	Methodological workflow for dataset, model, nutrition, backend, and frontend integration	16
4.1	Layered architecture of the food recognition and nutrition system	24
4.2	Runtime request-response flow	25
4.3	Entity relationship used for class-to-nutrition lookup	27
5.1	Frontend detection overlay showing Bhatura and Chole detections . . .	35
6.1	Verified system-level coverage used for validation	37
6.2	YOLO11s final training curves from the resumed 72-class run	40
6.3	Merged YOLO11s training result curves for the complete 100-epoch run	40
6.4	After-discontinuation YOLO11s bounding-box evaluation curves	41
6.5	Normalized confusion matrix for the YOLO11s held-out test evaluation	42
6.6	Sample YOLO11s predictions from the held-out test split	42
6.7	Frontend workflow checks used during qualitative validation	46
A.1	Visual verification half-sheet 1	69
A.2	Visual verification half-sheet 2	70
A.3	Visual verification half-sheet 3	71
A.4	Visual verification half-sheet 4	72
A.5	Visual verification half-sheet 5	73
A.6	Visual verification half-sheet 6	74
A.7	Visual verification half-sheet 7	75
A.8	Visual verification half-sheet 8	76
A.9	Visual verification half-sheet 9	77
A.10	Visual verification half-sheet 10	78

A.11 Visual verification half-sheet 11	79
A.12 Visual verification half-sheet 12	80
A.13 Visual verification half-sheet 13	81
A.14 Visual verification half-sheet 14	82
A.15 Visual verification half-sheet 15	83
A.16 Visual verification half-sheet 16	84

List of Tables

2.1	Condensed summary of reviewed research papers	13
3.1	Source datasets used for merged Indian food dataset	17
3.2	Examples of raw-to-canonical label normalization	18
4.1	Runtime nutrition database tables	26
4.2	Implemented backend endpoints	28
5.1	Important implementation files	36
6.1	Generated dataset summary	38
6.2	Final YOLO11s training setup	39
6.3	Final YOLO11s aggregate detector metrics	40
6.4	Visual class verification result	43
6.5	Nutrition database validation result	44
6.6	Nutrition mapping coverage for 72-class dataset	44
6.7	Verified supplemental nutrition rows	45
6.8	Corrected nutrition lookup examples	45
7.1	Representative categories in the 72-class dataset	49
7.2	Examples of improved nutrition mappings	51
A.1	Full 72-class dataset to INDB mapping table	65
A.2	Sample-level annotation issues found during visual verification	68
A.3	Per-class dataset counts after merge and train augmentation	85
B.1	Weight-loss macro targets for 2,000 kcal	91
B.2	Muscle-gain macro targets for 2,000 kcal	92
B.3	Maintenance macro targets for 2,000 kcal	92
B.4	Endurance macro targets for 2,000 kcal	93

ABBREVIATIONS AND NOTATIONS

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BMR	Basal Metabolic Rate
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
FYP	Final Year Project
HTTP	Hypertext Transfer Protocol
INDB	Indian Nutrient Database
IoU	Intersection over Union
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NMS	Non-Maximum Suppression
ORM	Object Relational Mapping
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
YOLO	You Only Look Once

Notations

B	Bounding box represented as center coordinates, width, and height.
C	Set of canonical food classes in the merged dataset.
P	Predicted probability or confidence score produced by the detector.
IoU	Area overlap ratio between predicted and ground-truth bounding boxes.
M_c	Nutrition mapping selected for class c .
E	Energy value in kilocalories per 100 grams.

Chapter 1

INTRODUCTION

1.1 Overview

Food logging appears simple at first, but it becomes inconvenient when it has to be done every day. A person has to remember what was eaten, estimate the quantity, search for a matching food entry, and repeat the same process for each item in the meal. In Indian meals this task is even less direct. A single plate may contain rice, dal, chutney, curry, a fried snack, and a sweet in small portions. The user may remember the meal clearly, but most logging applications still expect every item to be searched and entered separately.

Computer vision can reduce this effort, although it does not solve the complete nutrition problem on its own. Food does not have a fixed shape, lighting changes its appearance, and many items are partly hidden by bowls, plates, or other food items. Indian food adds another difficulty because many dishes share ingredients and colours. For example, `aloo_gobi`, `dum_aloo`, and `aloo_masala` can all contain potato-heavy textures, while chutneys may appear only as small side portions. During development it became clear that a useful system cannot stop after producing a confident visual label. That label must also be connected to a nutrition record that is reasonable for the dish.

This project builds a food recognition and nutrition-assistance prototype for Indian meals. The final system uses a YOLO object detector, a curated Indian food dataset, a SQLite nutrition database derived mainly from the Indian Nutrient Database, a FastAPI backend, and a Next.js frontend. After a user uploads a meal image, the backend detects visible food items, converts each detected class into a nutrition lookup, calculates macronutrients, and prepares suggestions using the user's daily goal and selected health context.

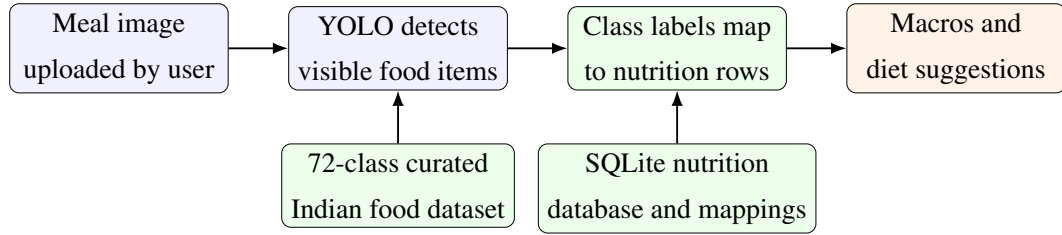


Figure 1.1: End-to-end workflow of the implemented food recognition and nutrition system

1.2 Motivation

The motivation for this work came from the limitations seen in normal food-logging workflows. Food entry has to be quick enough for daily use; otherwise, users stop maintaining the log after a few days. Indian food recognition also needs labels that match the cuisine. A generic model may identify a plate, bowl, or snack-like object, but it will not consistently separate `rajma_curry`, `sambar`, `poha`, `thepla`, or `bhakarwadi`. The recommendation part also needs user context. The same meal may be acceptable for a muscle-gain goal but unsuitable for weight loss, and a carbohydrate-heavy meal needs a different explanation for a diabetic user than for a user training for endurance.

Another motivation became clearer only after implementation started. Detection and nutrition lookup are related, but they are not the same problem. A detector may correctly return `samosa` or `palak_paneer`, while the system still has to connect that label to the correct database row. If the mapping is wrong, the interface can show calorie and macro values that appear precise but are actually misleading. In early tests, weak fuzzy matching produced cases such as `palak_paneer` being linked to a generic cottage-cheese entry instead of spinach paneer. After this, nutrition mapping was treated as a central part of the project rather than a small final step.

1.3 Problem Statement

The problem addressed in this project is to design and implement a full-stack system that can:

1. Detect one or more Indian food items from an uploaded image.
2. Normalize detected class labels into stable canonical food names.
3. Map detected foods to nutrition database records with maximum safe coverage.

4. Estimate calories, protein, carbohydrates, and fat per detected item.
5. Provide personalized dietary recommendations using user goals and health conditions.
6. Present the results through an accessible web interface with bounding boxes, serving adjustment, and macro tracking.

The problem is therefore wider than model training. It includes dataset engineering, data validation, backend API design, frontend interaction design, and nutrition-aware application logic. Each part affects the final reliability. A good frontend cannot fix a wrong food-to-nutrition mapping, and a trained detector becomes less useful if the class labels are inconsistent.

1.4 Objectives

The main objectives of the project are:

- To build a canonical Indian food dataset by merging multiple YOLO-format source datasets.
- To resolve duplicate and inconsistent food labels into a stable set of classes.
- To integrate a YOLO-based detection service into a FastAPI backend.
- To create a nutrition lookup service backed by a local SQLite database.
- To improve nutrition mapping quality through explicit class-to-database mappings.
- To implement macro calculation for different dietary goals and health profiles.
- To develop a Next.js frontend that supports image upload, detection visualization, nutrition cards, and recommendation display.
- To validate the dataset, class names, nutrition mappings, and service behavior.

1.5 Scope of the Project

The implemented prototype is limited to image-based recognition of Indian food classes that are present in the curated dataset. The model output is treated as a dish label, and nutrition values are reported per 100 grams unless the user changes the serving

assumption in the frontend. The system does not estimate physical food volume or exact portion size from one photograph. This boundary was kept deliberately because reliable portion estimation would require depth information, a known reference scale, segmentation, or controlled image capture. The prototype therefore concentrates on a practical recognition-to-nutrition workflow that can later be extended.

The backend and frontend are designed for local full-stack deployment. The backend exposes REST-style endpoints for image analysis, macro calculation, validation, and recommendation generation. The frontend is built as a single-page workflow for uploading images and reviewing results. The recommendation module uses Groq-backed language model generation when a key is configured, but the numerical nutrition calculations remain inside the backend. This separation was kept so that recommendations do not become the source of calorie or macro values.

1.6 Major Contributions

The major contributions of this project are summarized below. They are written from the implemented system rather than only from the model-training stage, because the final output depends on the complete pipeline.

1. A merged Indian food dataset containing 72 canonical classes and 48,693 exported image-label pairs.
2. A normalization pipeline that maps raw labels such as AlooGobi, aloo gobhi, thengai chutney, satham, and medu vadai into stable canonical labels.
3. Visual class verification artifacts showing real image samples for all 72 classes.
4. A nutrition database build process that imports INDB nutrition fields and creates a runtime SQLite database.
5. A safer nutrition mapping strategy that covers all 72 dataset classes using explicit semantic mappings and verified supplemental nutrition rows before fuzzy matching.
6. A FastAPI analysis endpoint that combines detection results, bounding boxes, nutrition data, and food-not-found handling.
7. A frontend interface that combines upload, detection visualization, serving adjustment, macro progress, and recommendations.

1.7 Report Organization

The report follows the same broad order as the project work. Chapter 2 reviews the supplied research papers and identifies the gap addressed by the implementation. Chapter 3 explains dataset construction, nutrition mapping, macro logic, and validation strategy. Chapter 4 presents the system architecture. Chapter 5 records implementation details and development challenges. Chapter 6 covers experiments and validation. Chapter 7 presents the results, and Chapter 8 discusses their meaning and limitations. Chapters 9 and 10 contain the conclusion and future work. The appendices include class mappings, API examples, and supporting verification material.

Chapter 2

LITERATURE REVIEW

2.1 Scope of the Review

This review uses the supplied research papers as background for the implemented project. The papers cover food recognition, Indian cuisine datasets, nutrition databases, semantic mapping, and recommendation systems. They were not read only as separate benchmarks; they were studied to understand what would be needed to move from an uploaded food image to a usable nutrition output in a working application.

For this project, the papers fall into three useful groups. The first group discusses food localization and classification through YOLO, CNNs, transfer learning, segmentation, and compact recognition models. The second group deals with nutrition databases and the harder-than-expected task of connecting a visual food label to a food-composition row. The third group looks at recommendation systems and LLM-assisted meal planning, including the risk of treating generated nutrition estimates as exact values.

2.2 Expanded Paper Summaries

2.2.1 Vision and Object Detection

2.2.1.1 IndianFoodNet

IndianFoodNet is relevant because it treats Indian food recognition as an object-detection task rather than only an image-classification task [1]. The use of YOLO-based bounding boxes is suitable for Indian meals, where chutney, dal, rice, fried snacks, and curries may appear together in one image. A single image label would miss these smaller or side items.

The work helped justify the use of a single-stage detector in the project architec-

ture. In this system, YOLO11s first localizes food items, and the backend then performs nutrition lookup for each detected class. IndianFoodNet mainly focuses on the visual detection stage. The current project continues after detection by adding database-backed macro values and recommendation output.

2.2.1.2 Khana Dataset

The Khana dataset paper addresses the shortage of large Indian cuisine benchmarks by presenting more than 131,000 images across 80 classes [2]. It also makes clear why Indian food recognition is difficult: several dishes share colour, texture, garnish, and serving style even when their ingredients and macro values are different.

This point matched the problems seen while merging the project dataset. Canonical class normalization, train-only augmentation, and visual class verification were needed because a confused label does not only affect detector accuracy; it also affects the nutrition record selected later. The paper also shows why class-wise checking is useful, since aggregate metrics can hide weak classes.

2.2.1.3 Enhanced Food Image Recognition

The enhanced food image recognition study uses transfer learning with lightweight networks such as MobileNetV2 to keep inference cost low while maintaining acceptable precision [3]. In food logging, this is a practical requirement because users expect a result soon after taking or uploading a meal image.

The project uses YOLO detection rather than single-image classification, but the same response-time concern remains. The backend has to receive the uploaded image, run inference, attach nutrition data, and return a response without feeling slow. For that reason, model size and service design were treated as deployment concerns, not only as training choices.

2.2.1.4 Single-item Training for Multi-dish Recognition

The single-item training paper studies complex Indian platter recognition using a Squeeze-and-Excitation DenseNet121-style architecture with class-agnostic feature extraction [4]. The reported accuracy is high, but the pipeline is heavier and more complex to deploy in a simple web application.

This project keeps the recognition path simpler. YOLO performs localization and classification in one inference pass, which made it easier to integrate with the FastAPI backend. This choice does not solve every problem caused by overlapping foods, but it

was more practical for the prototype than building a segmentation-first pipeline.

2.2.1.5 Lightweight and Parameter-Optimized CNN

The lightweight CNN paper concentrates on latency reduction for real-time food calorie estimation [5]. It adjusts convolution, pooling, and dropout choices to make prediction faster while keeping accuracy usable. The main lesson for this project is straightforward: a meal-logging tool has to respond quickly enough for repeated use.

For this project, the paper justifies compact detector variants and a backend where image inference, nutrition lookup, and recommendation generation are separate stages. Each stage can then be checked and improved independently. The main limitation from the project viewpoint is that the food coverage is mostly generic rather than focused on Indian dishes.

2.2.1.6 What is in an Indian Thali?

The Indian Thali paper studies one of the most difficult cases for Indian food recognition: overlapping and partly hidden items in a thali meal [6]. It uses segmentation, pixel masking, and a multi-camera setup to separate items on the same plate.

The paper is useful because similar overlap also appears in ordinary meal photographs. However, its capture setup is not convenient for a daily food-logging user. The project works from a single uploaded image and uses YOLO bounding boxes instead of multi-camera segmentation. The boundaries are less precise, but the method fits the web-based prototype better.

2.2.2 Nutrition Databases and Semantic Mapping

2.2.2.1 Development of INDB

The Development of an Indian Food Composition Database paper is central to the nutrition part of this project [7]. It reports nutrient values for cooked Indian recipes, which are more relevant for meal tracking than raw ingredient values alone. Cooking changes the final macro profile through oil absorption, water loss, ingredient ratios, and preparation method.

The backend converts the available INDB spreadsheet into a local SQLite database. This gives the application a fixed source for calories, protein, carbohydrates, and fat. The detector is used to identify the dish, while the database supplies the numerical macro values. This avoids asking a language model to guess nutrition values directly

from an image.

2.2.2.2 Indian Food Composition Tables

The Indian Food Composition Tables provide a strong analytical baseline for raw Indian foods [8]. They are valuable because they come from standardized biochemical measurements and help explain the nutrient profile of ingredients.

For this system, the limitation is that raw food values do not always match cooked meals. A curry, sweet, or fried snack can differ a lot from its raw ingredients after cooking. For this reason, the project uses cooked recipe-style INDB data as the main runtime source while still treating food-composition tables as an important reference.

2.2.2.3 Synergistic Frameworks for Indian Food Computing

The synergistic frameworks paper points out the mapping gap between visual food labels and nutrition database rows [9]. A detector may use one spelling, while the database may use a regional name or a longer recipe name. Without a mapping layer, even a correct visual label can fail during nutrition lookup.

The same issue appeared during implementation. Labels such as `satham`, `medu_vadai`, and `thengai_chutney` had to be normalized before they could be connected to database entries. The backend therefore uses canonical labels, explicit mappings, aliases, and conservative fuzzy matching rather than depending on exact string equality.

2.2.2.4 Nutrition Information Estimation from Food Photos

The nutrition information estimation paper combines food or ingredient predictions from photographs with structured nutrition datasets [10]. This is close to the logic used here because nutrition is not treated as a direct pixel-to-calorie regression problem.

The project uses dish-level detections instead of ingredient-level mass estimation. This boundary was kept because exact food weight cannot be recovered reliably from one image without depth, scale, or controlled capture conditions. The system therefore reports per-100g values and allows serving adjustment in the interface.

2.2.2.5 Label AI

Label AI maps barcode-scanned packaged foods to nutrition values and fitness goals [11]. That approach works well when the product already has a barcode and a standardized label. The identifier is clear, so nutrition lookup is comparatively direct.

Home-cooked Indian meals usually do not provide such identifiers. A plate of poha, rajma, idli, dosa, or thali has to be recognized visually. The present system replaces barcode input with YOLO detection and then connects the detected class to a SQLite nutrition database. This comparison explains why a visual-to-database mapping layer is needed for cooked food.

2.2.3 Recommendations, LLMs, and System Integration

2.2.3.1 Diet Engine

Diet Engine combines YOLOv8 food detection with an NLP chatbot for real-time dietary guidance [12]. It is one of the closer works to this project because it connects object detection with advice instead of stopping at classification.

The system follows a similar broad loop but adapts it for Indian foods. The dataset, class names, and nutrition mappings are domain-specific, and the recommendation layer receives structured food and macro data from the backend. As a result, the advice is connected to the detected meal rather than only to typed user input.

2.2.3.2 AI-driven Personalized Meal Planning

The AI-driven personalized meal planning paper presents a web platform that uses generative AI for diets based on user goals and health constraints [13]. Its relevance is partly technical and partly practical, because the user needs one place to handle goals, health conditions, and dietary suggestions.

This project adopts the same full-stack view through a Next.js frontend and FastAPI backend. The difference is that the recommendation begins with an uploaded meal image. Detected foods and calculated macros are passed to the recommendation layer so that the advice is tied to what the user actually logged.

2.2.3.3 NutriGen

NutriGen shows that LLMs can generate personalized meal plans when they are constrained by nutrition databases and user preferences [14]. The useful idea is not just the use of an LLM, but the way structured nutrition context is provided to it.

This shaped the recommendation service design in the project. The backend calculates calories and macros first, then passes structured information to the recommendation layer. The LLM is therefore not used as the source of exact numerical nutrition values, which reduces the chance of unsupported calorie or macro claims.

2.2.3.4 LLMs and Computer Vision for Personalized Nutrition Advice

The LLM and computer vision nutrition paper combines OCR with language models to read printed food labels and generate dietary advice [15]. It follows a useful pattern: computer vision first extracts grounded information, and language generation is applied afterward.

The present system follows that pattern, but the grounded input is cooked-food detection instead of OCR. Most Indian meals do not have printed labels, so the meal image itself becomes the input. YOLO detections and database-backed macros provide the structured context for the recommendation service.

2.2.3.5 AI-Based Nutrition Recommendation System

The AI-based nutrition recommendation system uses combinatorial logic and user profile constraints to generate diet plans within macronutrient boundaries [16]. This is useful because it keeps recommendations within explicit nutrition limits instead of producing open-ended advice.

From this project's perspective, the limitation is that such a planner does not verify the meal the user actually ate. The system adds a visual logging step before recommendations are produced, creating a clearer connection between the uploaded meal and the advice shown to the user.

2.2.3.6 Evaluation of ChatGPT for Nutrient Estimation

The ChatGPT nutrient-estimation evaluation reports that vision-language models can identify foods reasonably well but still struggle with exact macronutrient estimation from meal photographs [17]. The concern is that fluent text can make uncertain estimates appear more reliable than they are.

The implemented architecture avoids this problem by keeping numerical nutrition calculation outside the LLM. Calories, protein, carbohydrates, and fat come from the SQLite nutrition database and mapping table. The LLM receives already-computed values and is used only for explanation and recommendation text.

2.2.3.7 Visual Nutrition Analysis

The visual nutrition analysis study uses segmentation models such as UNet or DeepLab-style networks to isolate food pixels and estimate nutrition through regression [18]. Such an approach can work well when segmentation masks are accurate and the dataset

contains detailed nutrition labels.

This project does not estimate nutrients directly from pixels. It separates recognition from nutrition lookup: YOLO identifies the food class, and the backend retrieves macros from a database. The design was easier to validate within the project scope, especially because exact serving-size estimation was not included.

2.3 Comparative Summary

Table 2.1 gives a short comparison of the reviewed papers. The earlier summaries explain the connection between each paper and the implemented system.

Table 2.1: Condensed summary of reviewed research papers

S. No	Technique & Reference	About Technique	Applications	Dataset Used	Results	Limitations
1	YOLO architectures, IndianFood-Net [1]	Bounding-box detection.	Real-time Indian food tracking.	IFN: 5,500+ images.	YOLOv5 mAP 0.807; YOLOv8 0.671.	No nutrition database.
2	CNNs, ViT and ConvNeXT, Khana [2]	Fine-grained classification.	Indian cuisine benchmarking.	Khana: 131K images, 80 classes.	ConvNeXT-S top-1: 86.72%.	Single-label task.
3	Transfer learning, Enhanced Food CNNs [3]	MobileNetV2 retraining.	Fast web/mobile classification.	Custom data.	Weighted precision: 76%.	No multi-dish detection.
4	SE-DenseNet121, single-item training [4]	SE feature extraction.	Cluttered platter recognition.	30 Indian categories.	Top-1: 97.48%; saved 333–500 annotation hours.	Computationally heavy.
5	Lightweight CNN [5]	Tuned MaxPool and Dropout layers.	Mobile calorie estimation.	Food-101 and Fruit-360.	About 85%; 0.0013 s.	Mostly Western foods.
6	Object segmentation, Indian Thali [6]	Pixel masking and KNN matching.	Occluded thali segmentation.	ITD and WED.	Segmented 5–10 overlapping items.	Needs five-camera rig.
7	Biochemical aggregation, INDB [7]	Cooked recipe macro profiling.	Indian recipe nutrition.	INDB.	Macros for 1,014 cooked recipes.	Needs AI mapping bridge.
8	Analytical baseline, IFCT [8]	Direct lab measurements.	Raw Indian food reference.	NIN databank.	Standardized 528 foods.	Raw foods only.
9	Fuzzy semantic mapping [9]	Partial string matching to database rows.	Visual-to-database mapping.	Visual and nutrition tables.	Supports 72-class linking.	Needs clean taxonomy.
10	Pretrained CNNs and DB lookup [10]	Visual prediction plus nutrition lookup.	Multi-task macro extraction.	Yelp and Nutrition5k.	85% ingredient accuracy; F1 49.09%.	Multiple lookup overhead.
11	CatBoost and barcodes, Label AI [11]	Barcode APIs and ML scoring.	Packaged food tracking.	Open Food Facts.	R^2 0.9002; explained variance 0.9010.	Not for home-cooked meals.

S. No	Technique & Reference	About Technique	Applications	Dataset Used	Results	Limitations
12	YOLOv8 and NLP bot, Diet Engine [12]	Detection piped to chatbot.	Mobile diet consulting.	Custom: 6,692 images.	F1 0.86 at 0.264 confidence; mAP 0.897.	Weak Indian generalization.
13	LLM meal planning [13]	DeepSeek/ChatGPT diet generation.	Chronic disease diets.	User health profiles.	90% weight-loss accuracy; 87% diabetic alignment.	Manual inputs.
14	Prompted LLMs, NutriGen [14]	USDA-constrained generation.	Personalized plans.	USDA database.	Calorie error as low as 1.55%.	Requires manual logging.
15	OCR and LLM [15]	Printed label extraction for LLMs.	Context-aware diet chat.	Packaged labels.	100% text extraction for JSON output.	Cannot read cooked food pixels.
16	Combinatorial logic recommender [16]	Macro-boundary matching.	Seasonal diet generation.	Mediterranean database.	100% acceptable spring/summer plans for 840 users.	No visual verification.
17	VLM estimation, ChatGPT evaluation [17]	GPT-4V estimates macros from photos.	Zero-shot dietary assessment.	114 meal photos.	Mean correlation 0.62; exact match 33.3–57.9%.	Hallucinates exact math.
18	UNet segmentation, Visual Nutrition Analysis [18]	Segmentation and nutrient regression.	Direct macro estimation.	Nutrition5k.	Dice 97.69%; Jaccard 95.52%.	Hidden ingredients remain difficult.

2.4 Research Gap

The reviewed papers show that food recognition, nutrition lookup, and recommendation generation are often handled separately. Detection papers can localize foods, but they may not connect those labels to reliable nutrition records. Nutrition databases contain structured macro values, but they do not identify what is present in a user-uploaded image. Recommendation systems can generate diet suggestions, but they often depend on manual inputs and may not verify the meal visually.

This project addresses that integration gap for Indian food. The system combines YOLO-based detection, a canonical Indian food label set, SQLite-backed nutrition lookup, explicit class-to-database mapping, deterministic macro calculation, and structured recommendation generation. The workflow is more complete than a detector-only system and avoids asking an LLM or vision-language model to guess exact nutrition values from a photograph.

Chapter 3

METHODOLOGY

3.1 Methodological Overview

The methodology follows the order in which the project became workable. The dataset first had to be made consistent, because model integration is difficult to trust when the same food appears under different labels. The nutrition database also had to be checked before any recommendation output could be considered meaningful. After these two parts became stable, the backend and frontend could be tested with fewer unknowns. The workflow starts from YOLO-format food datasets and moves through class normalization, split generation, train-only augmentation, detector integration, nutrition import, class-to-food mapping, and finally API and interface development. Figure 3.1 summarizes this process.

This order was followed because early mistakes move forward into later stages. If one dish remains under two class names, the detector learns them as different classes. If a detector class is linked to a wrong nutrition row, the frontend may still display a clean-looking but incorrect result. For this reason, the methodology gives as much attention to data checking and mapping verification as it gives to training the model.

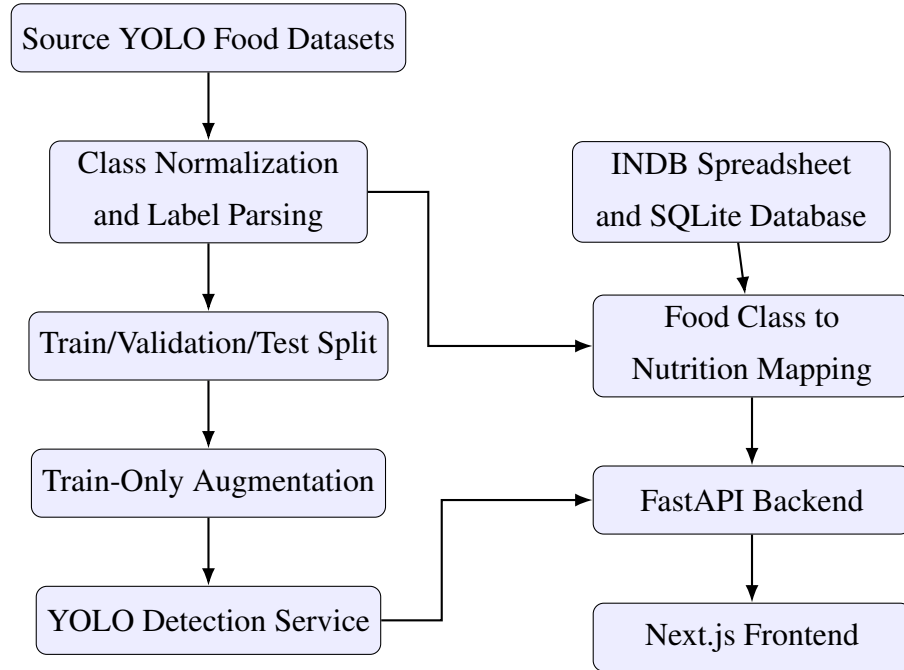


Figure 3.1: Methodological workflow for dataset, model, nutrition, backend, and frontend integration

3.2 YOLO Detection Approach

YOLO, short for You Only Look Once, is used because it treats object detection as a single-pass prediction task. For a food image, the model predicts bounding boxes, class probabilities, and confidence scores in one forward pass. This suits the web application because the backend needs to return results quickly enough for the user to inspect the uploaded meal without waiting too long.

The project uses the YOLO label format and inference style followed by recent Indian food detection systems such as IndianFoodNet and Diet Engine [1, 12]. During inference, the model returns candidate detections for visible food items. The backend applies confidence filtering and non-maximum suppression so that repeated boxes are removed before nutrition lookup. Each final detection contains a class name, confidence value, and normalized bounding box.

The reason for using YOLO is not speed alone. Indian meal photographs often contain more than one dish, and a whole-image classifier would usually ignore side items. YOLO lets the system detect multiple foods in the same image and map each detected class separately to the nutrition database. This also gives a useful debugging path: image region, detected class, database row, and final macro values can be checked one by one.

3.3 Source Dataset Collection

Five YOLO-format datasets were used as source datasets. The local dataset exports include Roboflow Universe metadata, dataset versions, source URLs, and CC BY 4.0 license notes [19–23]. Together, they contained 40,797 source images across training, validation, and test folders before project-specific merging. Combining them improved food coverage, but it also introduced naming and annotation issues. Some labels used CamelCase, some used spaces, some used regional words, and some referred to the same dish with different spellings. These differences were resolved before training so that the final model would not learn duplicate or confusing outputs. Table 3.1 summarizes the source data.

Table 3.1: Source datasets used for merged Indian food dataset

#	Source dataset	Classes	Train	Valid	Test
1	Indian food detection v1 [19]	17	7547	1053	453
2	Indian food v6 [20]	29	8088	213	105
3	Indian food v2 seven-class subset [21]	7	1698	159	92
4	IndianFoodNet YOLO export [22]	30	11387	1073	576
5	South Indian food detection v19 [23]	31	6478	1294	581

The raw class count was higher than the final class count because several labels referred to the same food. For example, `satham`, `rice`, and `WhiteRice` were normalized into `rice`. This was not only a naming cleanup. Object detectors learn one output channel per class, so equivalent labels left separate would train the model to treat the same food as different objects. The same normalized names later helped while preparing nutrition mappings.

3.4 Canonical Class Normalization

The canonical label set contains 72 food classes. The normalization process converts raw source labels into snake-case names, resolves spelling differences, and merges regional variants. Some cases still required manual checking. A script can clean punctuation and casing, but it cannot always decide whether two regional names refer to the same dish. Table 3.2 shows examples.

Table 3.2: Examples of raw-to-canonical label normalization

Raw variants	Canonical class
AlooGobi, aloo gobi, aloo_gobi	aloo_gobi
CoconutChutney, thengai chutney, coconut chutney	coconut_chutney
Idli, idly	idli
WhiteRice, rice, satham	rice
lemon satham	lemon_rice
medu vadai, medu vada	medu_vada

3.5 YOLO Label Representation

Each YOLO label file contains one row per annotated object:

$$y = (c, x, y, w, h) \quad (3.1)$$

where c is the class identifier, x and y are normalized center coordinates, and w and h are normalized width and height. The normalized coordinate system makes labels independent of image resolution. During parsing, each raw class identifier is first resolved to its source dataset class name and then converted to the canonical class name. This two-step conversion was necessary because class identifiers are local to each dataset and cannot be compared directly across sources.

3.6 Dataset Splitting and Augmentation

The merged dataset uses a 70/15/15 target ratio for training, validation, and testing. Multi-label images are represented through class-presence vectors. A stratified split is attempted first so that classes remain reasonably distributed across splits. When rare-label constraints make this infeasible, deterministic random splitting is used. The split stage was kept deterministic so that the same dataset can be regenerated and checked again.

Augmentation is applied only to the training split. This prevents transformed

training samples from leaking into validation or test evaluation. The augmentation strategy uses horizontal flipping and mild brightness and contrast changes. The intention is modest: increase coverage for underrepresented classes without producing unrealistic food photographs. Stronger transformations were avoided because colour and texture are important cues for visually similar Indian dishes.

Algorithm 1 Dataset merge and export procedure

```
1: Initialize empty sample list  $S$ 
2: for each source dataset  $D$  do
3:   Load source class names from data.yaml
4:   for each image-label pair in  $D$  do
5:     Parse YOLO labels
6:     Convert raw class names to canonical classes
7:     Add sample to  $S$ 
8:   end for
9: end for
10: Build multi-label matrix for all samples in  $S$ 
11: Split into train, validation, and test sets
12: Export images and labels using canonical class identifiers
13: Apply augmentation only to training samples below target count
14: Write final data.yaml, build log, and per-class count CSV
```

3.7 Nutrition Database Methodology

The nutrition database is built from the INDB spreadsheet. The required columns are `food_name`, `energy_kcal`, `protein_g`, `carb_g`, and `fat_g`. These columns are renamed to the runtime schema: `name`, `calories`, `protein_g`, `carbs_g`, and `fat_g`. A normalized food-name column is added for lookup and matching. Source metadata is also stored so that supplemental values can be separated from INDB rows.

The database contains two main tables. The first table, `indb_foods`, stores food records, macronutrients, and source information. The second table, `yolo_mappings`, stores precomputed mappings from YOLO classes to database food names. This separation keeps lookup fast and makes mapping decisions easier to inspect. It also allowed the mapping table to be regenerated during development without changing the original nutrition records.

3.8 Nutrition Mapping Methodology

Food-to-nutrition mapping is performed conservatively. Manual semantic mappings are attempted first by checking database food names and choosing the closest nutritionally meaningful record. Verified supplemental rows are used only when the available INDB sheet has no safe entry for a dataset class. Fuzzy matching is kept as a fallback and used only when the score is high enough. This caution is needed because two strings can look similar while the foods are nutritionally different, especially among sweets, curries, chutneys, and fried snacks.

This mapping step took more time than expected. Several detector labels were easy for a person to understand but did not match the database wording exactly. A regional or shortened class name could fail exact lookup, while fuzzy matching could select the wrong food because the words looked close. Since the final system displays numerical macro values, mapping was handled as a validation problem rather than a convenience feature.

For a detected class c , the mapping function can be represented as:

$$M_c = \begin{cases} \text{manual}(c), & \text{if a verified manual mapping exists} \\ \text{supplemental}(c), & \text{if a verified supplemental row exists} \\ \text{fuzzy}(c), & \text{if } \text{score}(c) \geq 0.78 \\ \emptyset, & \text{otherwise} \end{cases} \quad (3.2)$$

The current mapping covers all 72 dataset classes. The classes `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` were absent from the available INDB sheet, so they are handled through verified supplemental nutrition rows with source metadata rather than weak fuzzy substitutes.

3.9 Macro Calculation Methodology

The macro engine converts a daily calorie target into grams of carbohydrates, protein, and fat. Each goal starts with a base macro split, and health conditions modify this split before normalization. The calculation was kept rule-based and explicit so that it could be checked manually during testing. If T is the target calorie value and p_c , p_p , and p_f are the final percentages for carbohydrates, protein, and fat, then:

$$G_c = \frac{T \times p_c}{4 \times 100} \quad (3.3)$$

$$G_p = \frac{T \times p_p}{4 \times 100} \quad (3.4)$$

$$G_f = \frac{T \times p_f}{9 \times 100} \quad (3.5)$$

where G_c , G_p , and G_f are grams of carbohydrates, protein, and fat respectively. The values 4, 4, and 9 represent calories per gram for carbohydrates, protein, and fat. The generated values are later shown in the frontend as target macros against which the detected meal can be compared.

3.10 Validation Methodology

Validation is performed at multiple levels because one failing layer can make the final output misleading. Dataset validation checks class names, split counts, and visual class samples. Database validation checks row counts and schema. Mapping validation checks class-to-food mappings, coverage, and source provenance. Service validation checks whether the backend can initialize the nutrition service and return expected lookup results. Frontend validation checks whether the user workflow can display detections, nutrition labels, and recommendations. This layered approach made debugging manageable because problems could be isolated to the dataset, database, service, or interface instead of being treated as one large application error.

Chapter 4

SYSTEM DESIGN

4.1 Design Goals

The system was designed around five practical goals: usability, modularity, traceable data flow, local deployability, and extensibility. Usability means that a user should be able to upload a food image and understand the result without knowing the internal model details. Modularity keeps dataset building, model inference, nutrition lookup, recommendation generation, and interface rendering separated enough for independent debugging. Traceable data flow makes detected labels, confidence scores, bounding boxes, and nutrition sources visible instead of hiding them inside backend logic. Local deployability keeps the prototype easy to run on a developer machine, while extensibility leaves space for additional classes, improved models, and better nutrition sources later.

These goals came from the problems faced during development. Early testing showed that a food-recognition result is hard to trust when the label, confidence score, and nutrition source are not visible together. The design therefore avoids treating the detector as an unexplained black box. Each layer returns structured information that can be checked during development and shown clearly in the frontend.

4.2 High-Level Architecture

The project follows a layered architecture because each part has a different responsibility. The data layer contains the source datasets, merged YOLO dataset, INDB spreadsheet, and SQLite nutrition database. The AI layer contains the YOLO model and inference service. The backend layer exposes APIs and coordinates services. The frontend layer handles the user workflow. Figure 4.1 shows the high-level architecture.

This layering reduced debugging time. When a wrong macro value appeared,

the mapping table could be checked without retraining the detector. When the frontend display looked incorrect, the backend JSON response could be inspected separately. This separation made the project easier to handle as a full-stack application instead of a single large script.

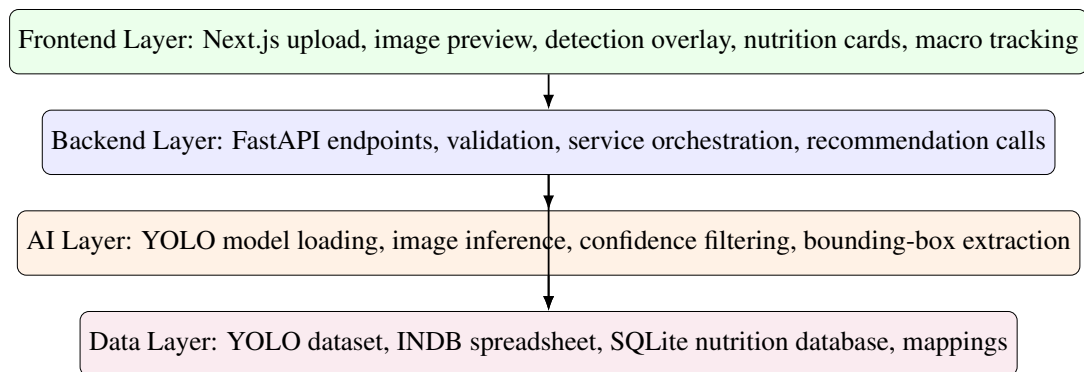


Figure 4.1: Layered architecture of the food recognition and nutrition system

4.3 Runtime Data Flow

The runtime flow begins when the user uploads an image. The frontend sends the raw image to the backend. The backend validates the upload, sends it to the vision service, receives detected objects, and looks up nutrition for each detected label. The response includes image dimensions, bounding boxes, labels, confidence values, macro values, and nutrition source metadata. The frontend then renders boxes over the image and displays nutrition cards, allowing the user to connect the visual prediction with the nutrition values.

The response format was kept detailed deliberately. During testing, it was not enough to know that the application detected a food item. It was also necessary to check where the box was drawn, how confident the model was, which database row was selected, and whether the macro values came from INDB or a supplemental source.

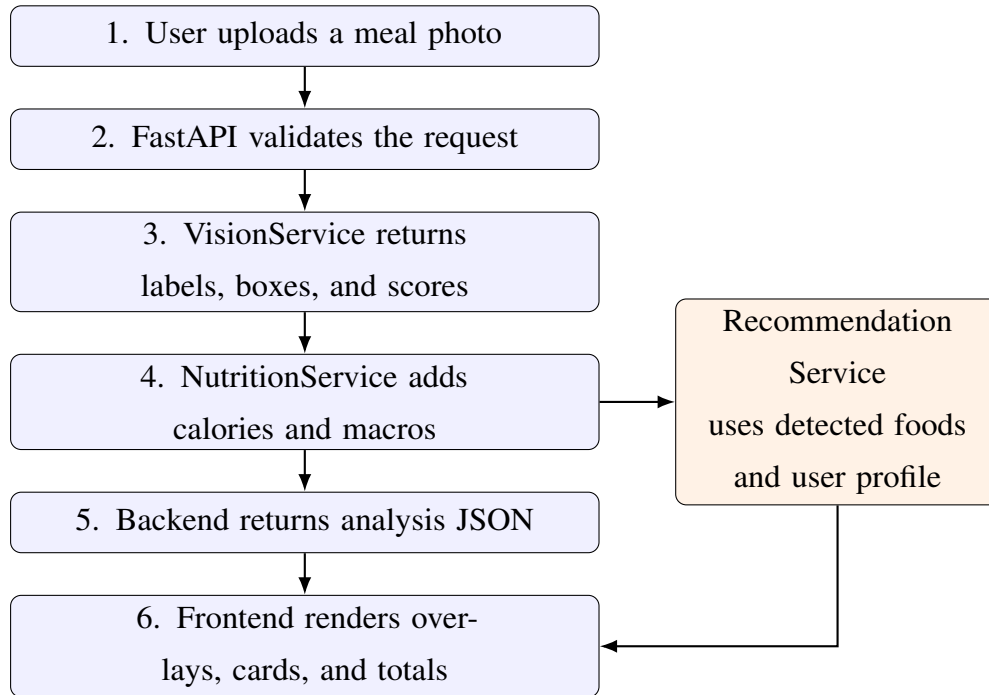


Figure 4.2: Runtime request-response flow

4.4 Backend Design

The backend is implemented with FastAPI, an ASGI-compatible Python framework suitable for typed request models, upload handling, and asynchronous service execution [24]. The backend contains the following core services:

- **VisionService:** loads the YOLO model and performs inference on uploaded images.
- **NutritionService:** connects to SQLite, caches food names, loads `yolo_mappings`, and returns macro values.
- **RecommendationService:** computes goal and health-condition context and generates dietary suggestions.

The backend design separates the endpoint layer from service logic. Nutrition lookup can be tested without running image inference, and macro calculation can be tested without uploading files. This separation was useful while debugging the nutrition layer because lookup errors could be reproduced with a single food name instead of repeating the complete image-upload workflow.

4.5 Frontend Design

The frontend is built with Next.js 14 and React [25]. It follows one main workflow: upload an image, analyze it, inspect detections, adjust serving estimates, review macro impact, and request recommendations. Frontend state is organized around the image analysis result, selected goal, target calories, selected health condition, and macro totals.

The interface opens directly to the working tool rather than a marketing-style page. The first screen contains upload controls, user nutrition context, and result panels. This fits the purpose of the application because it is meant to be used for checking a meal, not for browsing product information. During testing, the most useful interface behaviour was seeing the uploaded image and nutrition cards together, because this quickly showed whether the visual prediction and database output matched.

4.6 Database Design

The SQLite database contains a normalized nutrition table and a precomputed mapping table. Table 4.1 summarizes the database design.

Table 4.1: Runtime nutrition database tables

Table	Key columns	Purpose
indb_foods	id, food name, normalized name, calories, protein, carbohydrates, fat, source	Stores INDB and supplemental nutrition records.
yolo_mappings	YOLO class, matched food name, match score	Stores precomputed mappings from detector labels to database food rows.

The database is intentionally simple. A heavier database server would add setup work without improving the core prototype. Since the nutrition data is mostly read-only at runtime, SQLite is sufficient and keeps the project easy to run locally. It also makes the generated database file easy to inspect after rebuilding the mapping table.

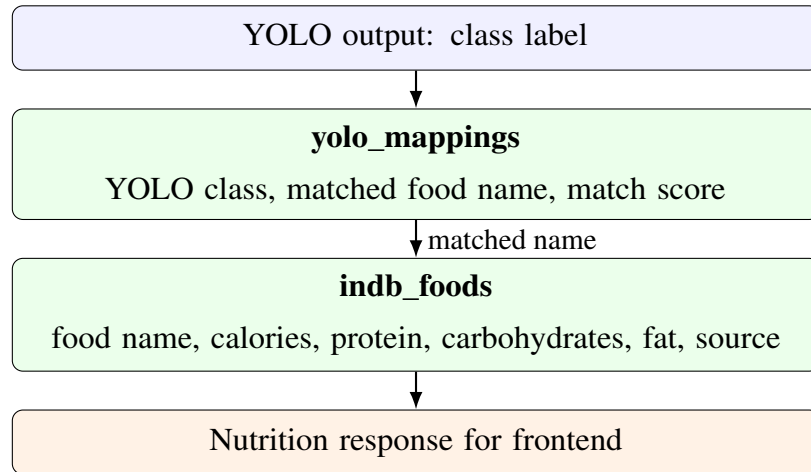


Figure 4.3: Entity relationship used for class-to-nutrition lookup

4.7 API Design

The API exposes endpoints for health checks, image analysis, nutrition testing, nutrition validation, macro calculation, goal metadata, health-condition metadata, and recommendations. The testing and validation endpoints were kept because they helped verify the system during development and can help future maintenance. Table 4.2 lists the implemented endpoints.

Table 4.2: Implemented backend endpoints

Endpoint	Method	Purpose
/	GET	Root API description and endpoint list.
/api/health	GET	Returns service health and model-loaded status.
/api/analyze-food	POST	Accepts image upload, runs YOLO inference, and attaches nutrition data.
/api/test-nutrition/ <food_name>	GET	Debug endpoint for a single food nutrition lookup.
/api/validate-nutrition	GET	Validates nutrition lookup coverage for model classes.
/api/user/ calculate-macros	POST	Calculates goal-specific macro targets.
/api/user/goals	GET	Returns supported dietary goals.
/api/user/ health-conditions	GET	Returns supported health condition options.
/api/recommendations	POST	Generates dietary recommendations from detected foods and user profile.

4.8 Security and Validation Design

The system validates upload type, empty uploads, maximum file size, and service readiness. The backend enforces a maximum image upload size of 10 MB. Rate limiting is implemented for the food analysis endpoint to avoid repeated heavy inference calls. Secrets such as the Groq API key are loaded from environment variables and are not written in the report.

These checks are basic, but they matter in a working prototype. Without them, a non-image file, an empty upload, or an unloaded model can produce unclear errors in the frontend. Returning structured failures makes the application easier to test and prevents the interface from showing incomplete nutrition results.

4.9 Design Trade-Offs

The design uses per-100-gram nutrition values instead of automatic portion estimation. This limits exact calorie accuracy, but it avoids pretending that one image is enough to recover food mass reliably. Monocular portion estimation would require plate scale, camera distance, depth, and food-volume assumptions. The frontend therefore uses serving multipliers so the user can adjust values when the visible quantity is clearly smaller or larger than the baseline.

The design also keeps explicit nutrition mappings in the database build process. This is less automatic than fuzzy matching, but it is safer for a nutrition-related application. A wrong nutrition value can be more harmful than a missing one. For that reason, classes without safe INDB rows are handled only through verified supplemental entries with source metadata. The prototype becomes a little more manual to maintain, but the output is easier to control.

Chapter 5

IMPLEMENTATION

5.1 Development Environment

The project was implemented as a full-stack application rather than only a training notebook. This made the work closer to a usable prototype, but it also meant that the detector, nutrition database, API response format, and frontend display had to stay consistent. A small class-name change in the dataset, for example, also required checking the nutrition mapping and display name. The backend uses Python, FastAPI, PyTorch, Ultralytics YOLO, Pillow, NumPy, Pandas, OpenPyXL, SQLite, and AioSQLite. The frontend uses Next.js, React, TypeScript, Tailwind CSS, and Lucide icons. The repository is organized into `backend/`, `frontend/`, `data/`, `models/`, `scripts/`, and `notebooks/` so that model assets, data processing, API code, and interface code remain separated.

5.2 Dataset Build Script

The dataset build script is located at `scripts/build_master_dataset.py`. It handles the work that would be difficult to repeat manually: source dataset scanning, class-name loading, YOLO label parsing, canonical class conversion, multi-label split creation, image export, training augmentation, and artifact writing. It writes three key outputs: `data.yaml`, `build_log.txt`, and `split_class_counts_before_after.csv`.

This script became one of the most important implementation pieces because the source datasets did not follow one naming convention. Some files had class names embedded in filenames, some labels had to be read through dataset-specific `data.yaml` files, and several raw names referred to the same dish. Automating the merge reduced manual copying mistakes and made it possible to rebuild the dataset after correcting normalization rules.

The final dataset contains 72 canonical classes. The generated balanced dataset contains 36,852 training image-label pairs, 5,922 validation pairs, and 5,919 test pairs. The total exported image-label pairs are 48,693. The larger secondary export retained in `data/master_dataset/` contains more aggressive train augmentation and is documented as an archived artifact.

5.3 Final YOLO Training Notebook

The final model training workflow is recorded in the final training notebook under `notebooks/`. The notebook is configured for Kaggle execution, loads the generated 72-class `data.yaml`, resumes the `yolo11s_indian_food` run from an existing checkpoint, and trains to 100 total epochs. The final resumed stage covers epochs 56 to 100 and uses YOLO11s with 640-pixel images, batch size 16, automatic mixed precision, checkpoint saving every 10 epochs, and plot generation enabled.

Training was resumed instead of restarted because the earlier checkpoint had already learned useful features from the merged dataset. The resumed run also provided later-epoch metrics through `results.csv` and training plots. The deployed model was not selected simply from the last epoch. The best exported weight was used because validation mAP reached its strongest value before the final epoch.

The run used PyTorch 2.10.0 with CUDA 12.8 on a Tesla T4 GPU. The exported artifacts include `best.pt`, `last.pt`, checkpoint weights, training curves, precision-recall curves, confusion matrices, prediction batches, and `results.csv`. The best model file was also exported as `yolo11s_indian_food_best.pt`, which is the detector artifact used for final evaluation.

5.4 Visual Class Verification Implementation

To verify that class names correspond to actual images, a visual verification script was created at `scripts/verify_food_classes_with_images.py`. The script groups exported images by the canonical class suffix in the filename, samples four images per class, parses corresponding labels for object crops, and writes contact sheets. This step was useful because file counts alone cannot show whether a class name visually matches the sampled images. The original eight sheets were split into sixteen half-sheets so that labels and samples remain readable in print. The full half-sheet set and the detailed annotation-issue table are placed in Appendix A to avoid repeating the same visual material in the implementation chapter.

The visual review confirmed that all 72 classes have actual image samples and that the class-name list is aligned. It also exposed a few dataset issues, such as a crop that did not match the expected food or a box placed on the wrong side item. These were not class-order failures, but they still matter because object detectors learn directly from the boxes provided during training.

5.5 Nutrition Database Build Implementation

The nutrition database build script is located at `backend/scripts/merge_db.py`. It reads the INDB spreadsheet, verifies required columns, normalizes food names, removes duplicate normalized names, converts nutrition columns to numeric values, and rebuilds the local SQLite nutrition database. The script then loads the current dataset classes and appends legacy model labels so that older deployed labels still resolve correctly.

Manual mappings are applied before fuzzy matching. This prevents incorrect string-based matches. For example, `aloo_gobi` maps to `Potato cauliflower (Aloo gobi)`, `palak_paneer` maps to `Spinach paneer (Palak paneer)`, and `rice` maps to `Boiled rice (Uble chawal)`. The script also adds source-backed supplemental rows for classes present in the detector dataset but absent from the available INDB sheet. This required more checking, but it avoided several wrong matches found in early tests.

5.6 Vision Service Implementation

The vision service loads YOLO model weights and processes uploaded images. It converts input bytes into an image representation, performs inference, applies class-level confidence rules, extracts bounding boxes, and returns structured detection dictionaries. Each detection contains the food label, confidence score, bounding box, and image dimensions.

The model is warmed up during backend startup with a dummy image. This reduces first-request latency, which otherwise can make the first upload feel unusually slow. The backend executes inference in a thread pool so that the asynchronous API loop is not blocked by model computation. Keeping the output as dictionaries also made it easier to attach nutrition data without changing the model inference code.

5.7 Nutrition Service Implementation

The nutrition service initializes by connecting to `data/nutrition.db`. It caches all food names and normalized names and loads the `yolo_mappings` table. Runtime lookup follows this order:

1. Check precomputed YOLO mapping.
2. Try exact normalized-name match.
3. Try partial normalized match.
4. Generate alias-based variations.
5. Use fuzzy matching as the last fallback.

The precomputed mapping step is the most important because it avoids incorrect fuzzy matches. The returned nutrition object includes display name, macro values, unit, source, and food-found status. During testing, this status field helped the frontend distinguish between a detected food with nutrition data and a detected food whose nutrition record was missing or uncertain.

5.8 Recommendation and Macro Implementation

The macro calculation function accepts a target calorie value, a selected goal, and a health condition. It applies base goal splits and health-condition modifiers, normalizes the result, and converts calories into grams. The recommendation service uses detected foods and user context to generate three practical suggestions. The prompt includes detected foods, calories, goal, health profile, and health-specific dietary guidance. The calculation was kept separate from recommendation text so that macro targets can be tested even when the external recommendation service is not configured.

5.9 Frontend Implementation

The frontend is implemented in `frontend/app/page.tsx` and reusable components. `ImageUpload` manages drag-and-drop upload and file preview. `ResultsPanel` displays detection cards, nutrition values, serving controls, macro totals, and recommendations. `NutritionLabel` displays calories, protein, carbohydrates, and fat. `ImageOverlay` draws bounding boxes over the uploaded image.

The serving controls were included because the detector can identify a food item but cannot know its exact mass from a single image. This keeps the interface honest: the system starts with a calculated baseline, and the user can adjust the assumed serving when the visible quantity is clearly larger or smaller.



Figure 5.1: Frontend detection overlay showing Bhatura and Chole detections

5.10 Code Organization

Table 5.1 summarizes the important implementation files.

Table 5.1: Important implementation files

File or component	Purpose
<code>backend/main.py</code>	FastAPI application, endpoints, startup sequence, request validation.
<code>vision_service.py</code>	YOLO model loading, inference, and detection formatting.
<code>nutrition_service.py</code>	SQLite lookup, mapping cache, and nutrition response creation.
<code>recommendation service</code>	Goal-aware and health-condition-aware recommendations.
<code>merge_db.py</code>	INDB import, supplemental nutrition rows, and YOLO-to-database mapping creation.
<code>food_normalizer.py</code>	Food-name normalization and variation generation.
<code>dataset build script</code>	Merged dataset construction and augmentation.
<code>Class verification script</code>	Contact-sheet generation for visual class verification.
<code>page.tsx</code> and UI components	Main frontend workflow, detection cards, serving controls, and macro display.

5.11 Implementation Challenges

The main challenges were label inconsistency, nutrition mapping ambiguity, and multi-component integration. Label inconsistency was handled through canonical class naming. Nutrition ambiguity was handled through explicit mappings and conservative fallback. Integration complexity was reduced by separating services and keeping JSON response shapes structured.

In practice, the mapping work needed more attention than expected because a visually correct food label can still produce the wrong macro values if it is linked to the wrong database row. Another challenge was keeping training artifacts, deployed model names, dataset classes, and frontend labels synchronized. Small mismatches did not always cause immediate failures, but they produced confusing results during testing. For this reason, validation scripts and debug endpoints were kept in the project instead of being removed after the first successful run.

Chapter 6

EXPERIMENTS AND VALIDATION

6.1 Experimental Scope

The experiments in this project are based on repository files, generated reports, and exported training artifacts. The evaluation covers dataset construction, YOLO11s detector training, validation and test metrics, class-name verification, nutrition database integrity, mapping coverage, backend service behavior, frontend workflow, and qualitative detection output. The final training notebook and exported result directory provide the detector metrics used for reporting mAP, precision, recall, confusion-matrix evidence, and prediction-batch samples for the latest 72-class model.

Validation was not limited to checking whether the model produced a score. Several parts of the system had to work together. A detection result was useful only if the class existed in the final `data.yaml`, the nutrition service could resolve the class name, and the frontend could display the returned macro fields correctly.

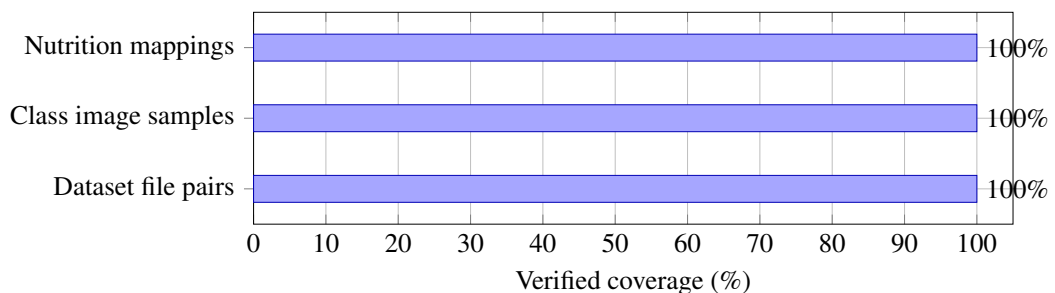


Figure 6.1: Verified system-level coverage used for validation

6.2 Dataset Validation

The first validation task checks whether the merged dataset exists, contains the expected split folders, and has a complete 72-class `data.yaml`. This step was necessary because a missing label file or class-count mismatch would make later training results unreliable. The generated dataset summary is shown in Table 6.1.

Table 6.1: Generated dataset summary

Property	Value
Canonical classes	72
Train images	36,852
Train labels	36,852
Validation images	5,922
Validation labels	5,922
Test images	5,919
Test labels	5,919
Total exported image-label pairs	48,693
Split target ratio	70/15/15

The split counts indicate that the generated dataset is complete: every exported image has a corresponding label file. Validation and test augmentation are disabled so that held-out samples remain real images rather than transformed versions of training data. As a result, the validation and test metrics are not based on augmented copies.

6.3 YOLO Training and Detector Evaluation

The final detector experiment used YOLO11s on the 72-class dataset. The run resumed from a previous checkpoint at epoch 56 and continued to 100 total epochs. This approach was used because the earlier checkpoint was already stable enough to continue, and restarting from the beginning would have increased training time without adding new data. Table 6.2 summarizes the final training setup recorded in the final training notebook and exported `args.yaml`.

Table 6.2: Final YOLO11s training setup

Property	Value
Model	YOLO11s object detector
Classes	72
Image size	640 pixels
Batch size	16
Training stage	Resumed from epoch 56 to epoch 100
Training time for resumed stage	10.636 hours
Hardware	Tesla T4 GPU, 15.64 GB VRAM
Framework	Ultralytics 8.4.60, PyTorch 2.10.0, CUDA 12.8
Best exported weight	yolo11s_indian_food_best.pt

The resumed training stage reduced training and validation losses with some normal epoch-to-epoch variation. The highest validation mAP@50 in `results.csv` was 0.83379 at epoch 78, while the final epoch recorded precision 0.8528, recall 0.8042, mAP@50 0.82806, and mAP@50:95 0.69164. Since the best validation value occurred before the final epoch, the exported best checkpoint was used for evaluation. Figure 6.2 shows the resumed-stage training curves. Figure 6.3 uses the merged result image to show the complete training trend after combining the available training segments.

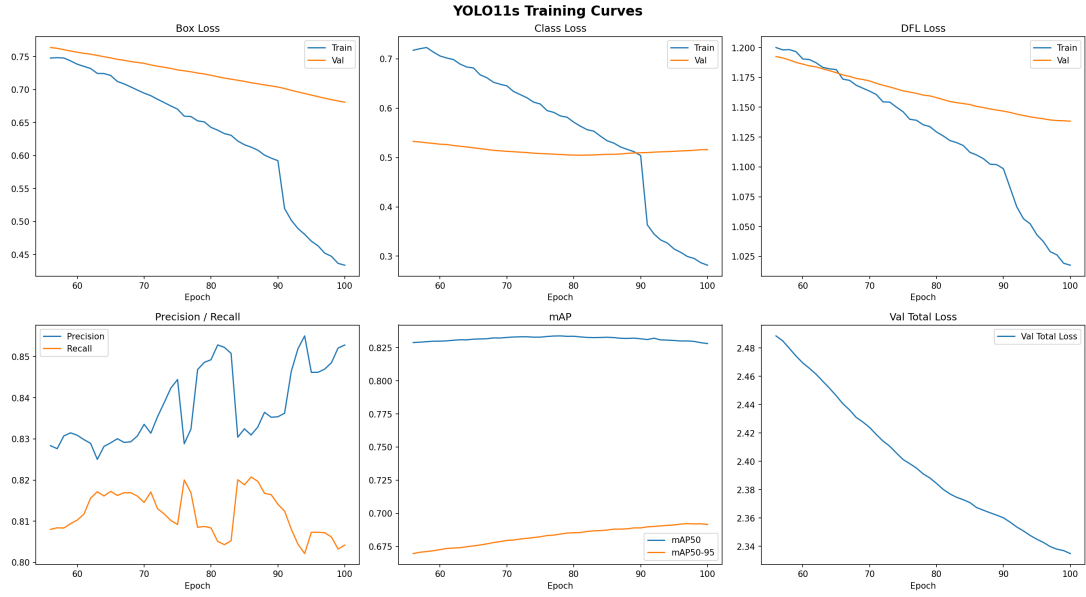


Figure 6.2: YOLO11s final training curves from the resumed 72-class run

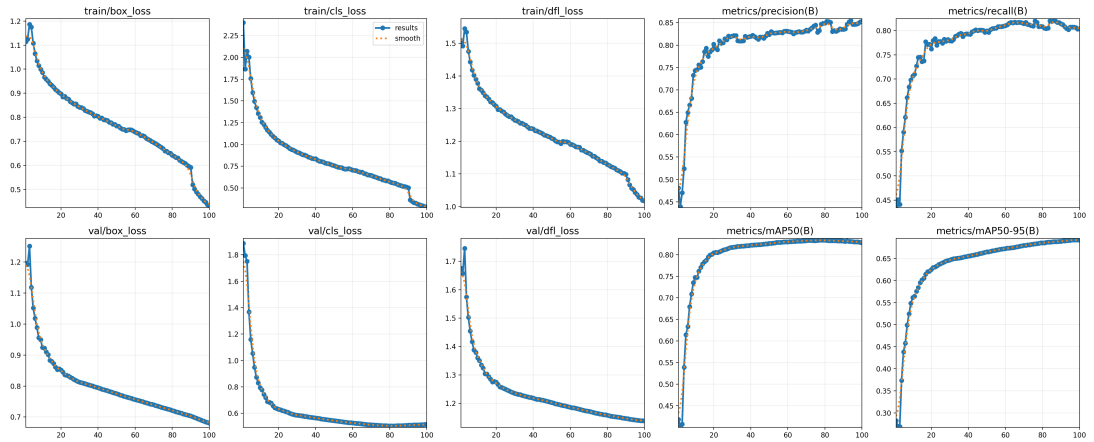


Figure 6.3: Merged YOLO11s training result curves for the complete 100-epoch run

The best exported model was evaluated on both validation and held-out test splits. Table 6.3 reports the aggregate detector metrics.

Table 6.3: Final YOLO11s aggregate detector metrics

Split	Images	Instances	Precision	Recall	mAP@50	mAP@50:95
Validation	5,922	8,812	0.847	0.807	0.830	0.692
Test	5,919	8,875	0.842	0.801	0.820	0.680

The validation run reported mAP@75 of 0.765, and the test run reported mAP@75 of 0.751. Both validation and test evaluation reported approximately

11.0 ms inference time per image on the Kaggle Tesla T4 environment, excluding application-level upload and response overhead. This means the reported inference time describes only the model stage; the full web application also includes file upload, image decoding, nutrition lookup, and frontend rendering.

The exported run also produced confidence and precision-recall curves for bounding-box evaluation. These plots helped during interpretation because they show how the model behaves across confidence thresholds instead of relying only on one aggregate value. Figure 6.4 includes the F1-confidence, precision-recall, precision-confidence, and recall-confidence curves from the after-discontinuation YOLO11s run.

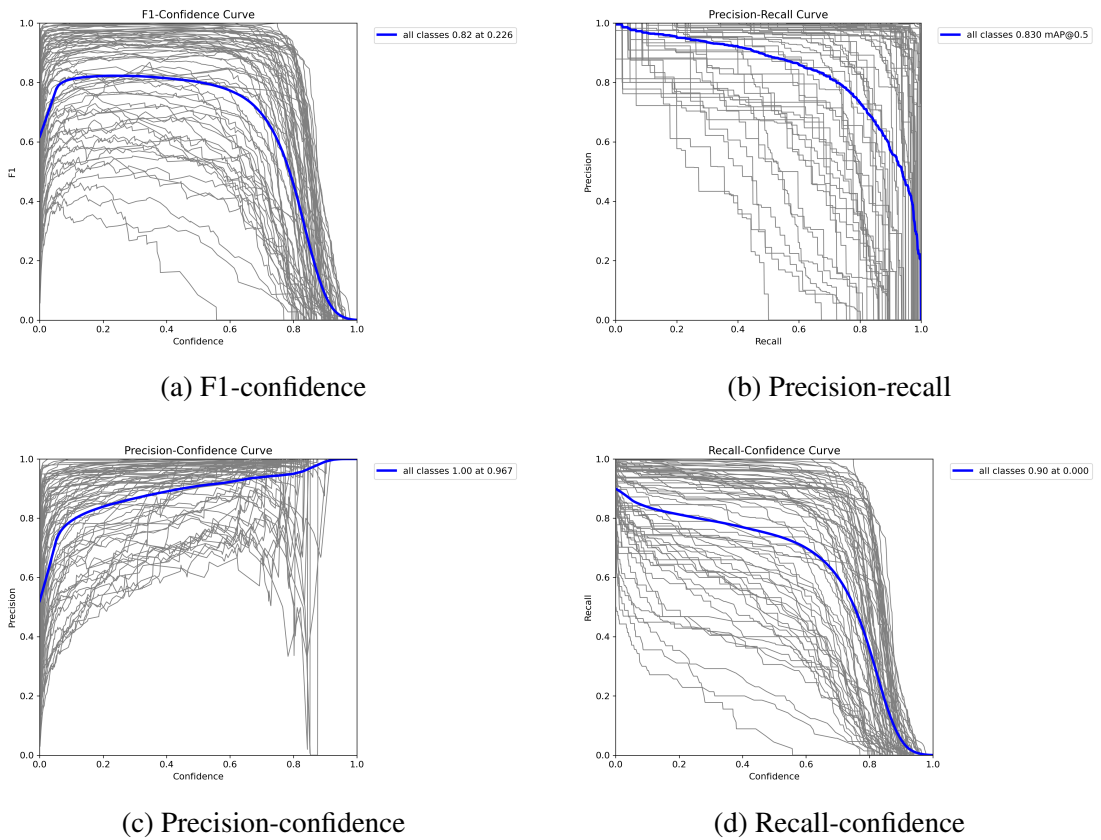


Figure 6.4: After-discontinuation YOLO11s bounding-box evaluation curves

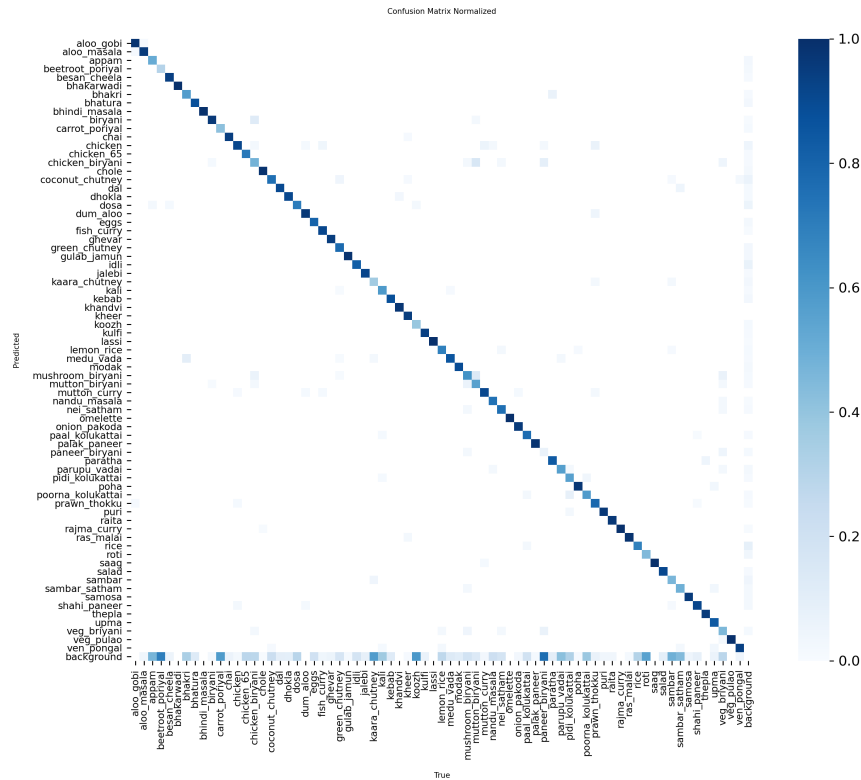


Figure 6.5: Normalized confusion matrix for the YOLO11s held-out test evaluation

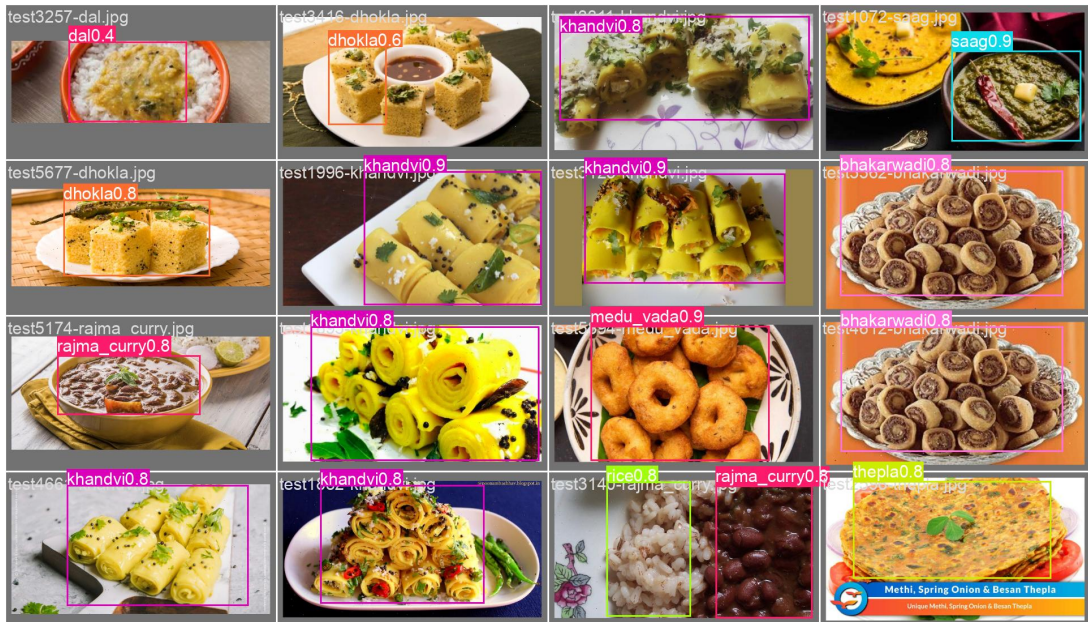


Figure 6.6: Sample YOLO11s predictions from the held-out test split

The aggregate metrics show that the final detector carries over from validation to the held-out test split with a small mAP drop. The normalized confusion matrix still shows uneven class behaviour, which is expected in a 72-class food detector where

several dishes look similar and some classes have fewer unique source images. In visual inspection, many mistakes were understandable when foods shared colour, texture, or serving style, such as rice-based dishes, paneer gravies, and chutney-side combinations.

6.4 Class Verification Experiment

The class verification experiment generated contact sheets with four images per class. The purpose was to catch practical mistakes that are easy to miss in code, such as class-order shifts, naming mismatches, or labels that do not match the visible food. The script found image samples for all 72 classes. Visual inspection found no class-list order problem. The issues observed were sample-level annotation issues rather than class-list errors.

This experiment was useful because it gave a quick human-readable check of the dataset. CSV counts alone would not show whether a jalebi sample actually looked like jalebi or whether a chutney box was placed on the correct food region. The contact sheets made these problems visible without opening hundreds of files manually.

Table 6.4: Visual class verification result

Check	Result
Classes checked	72
Classes with image samples	72
Classes without image samples	0
Contact sheets generated	8
Displayed samples per class	4
Class-order problem found	No
Sample-level issues found	3

6.5 Nutrition Database Validation

Nutrition database validation checks schema and row counts. The database contains 1,010 records in `indb_foods` and 103 records in `yolo_mappings`. The food table is based on INDB and five source-backed supplemental rows for classes absent from INDB.

The mapping count is larger than 72 because the table includes expanded dataset classes and legacy deployed-model labels for backward compatibility.

The row-count check was followed by lookup tests because a database can contain records and still return the wrong food for a detector label. This was especially important after the mapping logic was changed from mostly fuzzy matching to explicit class-to-food mappings.

Table 6.5: Nutrition database validation result

Table	Purpose	Count
indb_foods	INDB plus supplemental nutrition records	1,010
yolo_mappings	YOLO class to database mappings	103

6.6 Nutrition Mapping Coverage Experiment

The mapping coverage experiment checks how many of the 72 dataset classes have a safe nutrition mapping. The result is shown in Table 6.6. Coverage was measured on the final canonical class list rather than only on currently detected labels, because the model can return any of the 72 classes during use.

Table 6.6: Nutrition mapping coverage for 72-class dataset

Metric	Value
Total dataset classes	72
Mapped classes	72
Unmapped classes	0
Mapping coverage	100.00%

The classes `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` were not present as safe INDB rows, so verified supplemental entries were added with source metadata. The supplemental values use a FatSecret product nutrition label for `bhakarwadi` and Clearcals recipe nutrition pages for the remaining dishes [26–30]. This keeps the full 72-class dataset usable while avoiding unsafe fuzzy substitutions. These rows are documented as supplemental values and are not treated as equal to primary INDB coverage in the discussion.

Table 6.7: Verified supplemental nutrition rows

Class	Database row	kcal	Protein	Carbs	Fat
bhakarwadi	Bhakarwadi	510.0	10.76	57.03	26.55
ghevar	Ghevar	351.2	3.5	39.6	19.9
jalebi	Jalebi	316.8	3.4	44.6	13.8
khandvi	Khandvi	232.7	9.3	21.7	12.1
nandu_masala	Nandu Kari (Crab masala)	128.1	7.0	7.0	8.0

6.7 Service Lookup Validation

Service-level lookup validation was performed for classes that previously suffered from incorrect fuzzy matches. These examples were selected because they represented real failures observed during development, not only random test cases. The corrected results are shown in Table 6.8.

Table 6.8: Corrected nutrition lookup examples

Input class	Returned INDB display name
aloo_gobi	Potato cauliflower (Aloo gobhi)
palak_paneer	Spinach paneer (Palak paneer)
rice	Boiled rice (Uble chawal)
AlooGobi	Potato cauliflower (Aloo gobhi)
WhiteRice	Boiled rice (Uble chawal)

This confirms that the revised database mapping handles both canonical dataset labels and legacy deployed-model labels. It also confirms that the nutrition service can resolve common casing and naming differences such as AlooGobi and WhiteRice.

6.8 Backend API Validation

Backend validation focuses on service readiness and response structure. The analysis endpoint returns HTTP 503 if vision or nutrition services are unavailable. Empty

images, non-image MIME types, and uploads exceeding 10 MB are rejected. The analysis response includes food label, confidence, bounding box, nutrition source, macro values, and food-not-found status. This structured response is important because the frontend depends on predictable fields for visualization and macro tracking. It also makes debugging easier when a detection is correct but the nutrition object is missing or incomplete.

6.9 Frontend Workflow Validation

The frontend workflow was validated qualitatively by uploading sample food images and checking that the interface displays bounding boxes, detected labels, confidence values, nutrition cards, serving controls, and recommendation text. Figure 5.1 in Chapter 5 shows a successful example with Bhatura and Chole detections. The interface also handles no-food responses and backend error messages.

This workflow test was important because some issues appear only after the model response reaches the browser. Box coordinates must align with the displayed image size, serving changes must update macro totals, and recommendation text must use the current user goal rather than an old state.

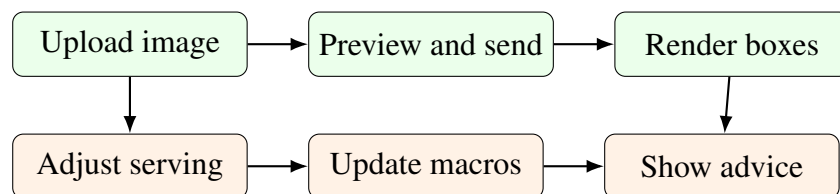


Figure 6.7: Frontend workflow checks used during qualitative validation

6.10 Macro Calculation Validation

Macro validation checks whether a target calorie value is converted correctly to grams. For example, for a 2,000 kcal weight-loss goal with a 40/35/25 split, the carbohydrate target is 200 g, protein target is 175 g, and fat target is approximately 56 g. Health profiles modify these base splits. The formula was discussed in Chapter 3. These calculations are simple enough to verify manually, which is useful because they directly affect the recommendation context shown to the user.

6.11 Experimental Findings

The experiments show that the dataset and application pipeline are internally consistent. The final YOLO11s model achieved 0.830 mAP@50 and 0.692 mAP@50:95 on validation, and 0.820 mAP@50 and 0.680 mAP@50:95 on the held-out test split. The system can detect food classes, map all 72 expanded dataset classes to nutrition records, and display results in the frontend. These results are suitable for a final-year prototype, while still showing practical limits such as class confusion among similar foods, dependence on per-100g nutrition values, and the difference between model inference time and full application response time.

Chapter 7

RESULTS

7.1 Dataset Engineering Results

The dataset engineering phase produced a merged dataset with 72 canonical classes. The main result was not only an increase in data size, but a cleaner label space built from five inconsistent sources. This mattered in later stages because the same canonical names were used in the YOLO training file, nutrition mapping table, backend response, and frontend display. Table 7.1 summarizes the types of classes represented in the final dataset.

Table 7.1: Representative categories in the 72-class dataset

Category	Example classes
Rice dishes	rice, lemon_rice, sambar_satham, veg_briyani, veg_pulao, ven_pongal
Flatbreads and fried breads	roti, paratha, bhakri, bhatura, puri, thepla
Curries and gravies	chole, rajma_curry, fish_curry, mutton_curry, shahi_paneer, palak_paneer
Snacks	samosa, onion_pakoda, medu_vada, parupu_vadai, bhakarwadi, khandvi
Sweets and desserts	gulab_jamun, ras_malai, kheer, kulfi, modak, jalebi
Chutneys and sides	coconut_chutney, green_chutney, kaara_chutney, raita, salad

The class verification output confirms that all classes have image samples. This

check was needed because a dataset configuration can list classes even when no usable labels exist for them. The verification step showed that this is not the case here. It also showed that the dataset is not perfect, since a few individual annotations were noisy. For a project prototype this was acceptable, but it is still recorded because annotation quality directly affects detector performance.

7.2 Detector Training Results

The final YOLO11s training run produced a usable 72-class detector. The model was trained through epoch 100, with the final resumed stage covering epochs 56 to 100. During that stage, the best validation mAP@50 recorded in `results.csv` was 0.83379 at epoch 78. The final epoch recorded precision 0.8528, recall 0.8042, mAP@50 0.82806, and mAP@50:95 0.69164.

The exported best model was evaluated separately on validation and held-out test splits. On validation, it achieved 0.847 precision, 0.807 recall, 0.830 mAP@50, and 0.692 mAP@50:95. On the held-out test split, it achieved 0.842 precision, 0.801 recall, 0.820 mAP@50, and 0.680 mAP@50:95. The small difference between validation and test mAP suggests that the model is not fitting only to the validation split. At the same time, the confusion matrix shows that some dishes remain difficult to separate, especially when they share colour, texture, or serving style.

7.3 Nutrition Mapping Results

The nutrition mapping results improved compared with the earlier deployed-model-only mapping. Previously, fuzzy matching produced incorrect mappings such as `aloo_gobi` to a cauliflower dessert and `palak_paneer` to a generic cottage-cheese dish. These errors were serious because the final output still looked numerical and confident even when the food record was wrong. The revised strategy maps these classes to more appropriate database entries and records source information for supplemental rows.

Table 7.2: Examples of improved nutrition mappings

Class	Corrected mapping	Reason
aloo_gobi	Potato cauliflower (Aloo gobhi)	Direct dish-level semantic match.
palak_paneer	Spinach paneer (Palak paneer)	Direct dish-level semantic match.
rice	Boiled rice (Uble chawal)	Best base record for plain rice.
sambar	Sambar	Exact database match.
shahi_paneer	Shahi paneer	Exact database match.

Some mappings remain approximate because the available INDB sheet does not contain every exact dish name. For example, `ven_pongali` maps to `Plain khitchdi`, and `chicken_biryani` maps to `Chicken pulao`. These assumptions are documented through confidence scores and macro verification reports rather than being hidden. For a nutrition prototype, showing where values come from is necessary because the output may influence diet choices.

7.4 Application Results

The frontend displays detected foods with bounding boxes and nutrition cards. In the demonstrated Bhatura and Chole image, the detector produced boxes around the large bhatura and the chole bowl. The frontend used these boxes for overlays and used the backend response to show macro values. This confirms the practical connection from image upload to visual output, nutrition lookup, and user-facing guidance.

This application result checks more than model accuracy. It verifies that an image can move through the upload flow, the backend can return structured detections, the nutrition service can attach macro values, and the frontend can render the combined result in a readable form.

7.5 Macro and Recommendation Results

The macro engine returns target grams for each dietary goal and health condition. For a fixed calorie target, different goals produce different distributions. A weight-loss goal

places more emphasis on protein, while an endurance goal increases carbohydrate share. Health profiles further adjust these splits. This gives the recommendation system more context than a generic food-recognition output.

The recommendation service produces three practical dietary suggestions based on detected foods and the user profile. For example, if the detected meal is high in fried carbohydrates and the user chooses weight loss, the recommendation can suggest smaller portions, additional protein, and lower-oil alternatives. If the user selects a diabetic profile, the advice can focus on carbohydrate moderation, fiber, and pairing with protein. The recommendation output was kept short because long diet explanations are less useful during meal logging.

7.6 Result Synthesis

Overall, the results show that the project is more than a detector experiment. The dataset, model, nutrition database, backend, and frontend connect into a working pipeline. The most important outcome is that visual detector output can be traced to explicit nutrition records and user-facing macro guidance. The results also show the boundaries of the prototype: portion size is still controlled by user adjustment, some food classes are naturally difficult to distinguish, and nutrition mappings depend on the availability of suitable food-composition records.

Chapter 8

DISCUSSION

8.1 Interpretation of Results

The results show that a food recognition system is useful only when detection is connected to the rest of the workflow. The detector tells the system what appears in the image, but the nutrition layer decides what that label means in calories and macros. The frontend then makes those numbers visible and adjustable. The main outcome of this project is therefore the integration of these layers, not only the training of a YOLO model.

The dataset engineering work was also important. A model trained on inconsistent labels would produce confusing outputs even if training completed successfully. Canonical class normalization gave the detector a stable label space and also made nutrition mapping possible. The visual verification sheets were useful because they exposed dataset issues that a file-count check would miss. A few noisy annotations remain, but the class list itself is aligned.

One practical observation from the project is that food recognition errors are not always clear from metrics alone. A small class-label error becomes more serious when it leads to a wrong nutrition record. For this reason, the results have to be interpreted as a complete chain from image to recommendation, not only as detector scores.

8.2 Strengths of the System

One strength of the system is its full-stack completeness. The project does not stop after model inference. It includes dataset building, backend APIs, nutrition database integration, recommendation logic, and a frontend workflow that can be used and inspected. This made the implementation more difficult, but it also made the final output

closer to a real application.

Another strength is domain specificity. The dataset focuses on Indian foods, and the nutrition database uses Indian food names and macro values. The alias and mapping logic also accounts for regional naming patterns, which matters for labels such as `satham`, `medu_vadai`, `thengai_chutney`, and `nandu_masala`.

The conservative mapping strategy is also a strength. Classes are mapped through explicit semantic rules first, and classes absent from INDB are covered only when a source-backed supplemental row is available. In a nutrition-related application, this reduces the risk of weak fuzzy matches producing misleading macro values.

The project is also documented through reproducible artifacts. It contains scripts for dataset construction, nutrition database creation, visual class verification, and macro verification. These scripts make the data pipeline inspectable instead of leaving the dataset and mappings as unexplained files. For a final-year project, this is useful because the work can be checked through generated artifacts rather than only through screenshots.

8.3 Limitations

The main limitation is portion size. The system reports macros per 100 grams and allows serving adjustment, but it does not estimate exact food mass from the image. This limits calorie precision. Accurate portion estimation would require depth information, reference objects, segmentation masks, or learned volume estimation.

Nutrition source consistency is another limitation. Five classes are covered through supplemental external sources because the available INDB sheet lacks safe equivalent entries. Some other classes use documented substitutes rather than exact dish rows. This is acceptable for a prototype, but a health-sensitive deployment would need stronger food-composition validation and expert review.

Dataset noise also remains. Visual verification found a small number of annotation issues. Large food datasets often contain such noise, but object detectors are sensitive to label quality. In this project, noisy samples did not prevent a working detector from being trained, but they should still be considered while interpreting the results.

The detector also behaves unevenly across visually similar dishes. The final YOLO11s metrics are suitable for a working prototype, but the confusion matrix shows that some classes are harder than others. This is especially important for a nutrition application because a wrong food class can propagate into an incorrect nutrition lookup even when the interface and database are functioning correctly.

The recommendation layer depends on an external language model service when

configured. This is acceptable for a prototype, but it makes recommendation output harder to validate than the rule-based macro calculation. A self-trained recommendation model with fixed input-output constraints would provide better control in a later version.

8.4 Ethical and Practical Considerations

Nutrition applications can influence user behaviour. Therefore, the system should avoid presenting estimates as medical advice. Recommendation text should be framed as general guidance, and users with medical conditions should consult qualified professionals. The application should also communicate uncertainty when a detected food uses supplemental nutrition data or when model confidence is low.

Data privacy is another consideration. Food images can reveal personal habits, location context, and health patterns. A production version should minimize image retention, protect API communication, and provide clear user consent. The current prototype is local and does not implement cloud storage for uploaded images.

There is also a practical responsibility in how results are displayed. A bounding box and macro card can make an output look certain even when the food class or serving size is uncertain. For this reason, confidence scores, nutrition sources, and serving controls are important parts of the interface.

8.5 Why Supplemental Rows Were Added

The classes `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` were absent from the available INDB file. Mapping them through fuzzy matching would be unsafe because a fried sweet should not be mapped to a generic sweet if the macro profile is different, and a crab masala class should not be mapped to a spice mix or unrelated seafood item. The database therefore adds verified supplemental rows for these classes and records their source URLs. This gives complete dataset coverage while keeping provenance visible.

This decision follows a rule used throughout the project: it is better to document an approximate or supplemental value than to hide a weak match behind a confident interface. In a nutrition application, transparency is more important than making every lookup appear equally reliable.

8.6 Lessons Learned

The project shows that applied AI systems require more than model selection. Dataset curation, naming conventions, data contracts, API design, error handling, and UI workflow affected the final system as much as the detector choice. Food recognition is especially dependent on domain knowledge because visual labels and nutrition records rarely use identical names.

Another lesson is that validation artifacts are valuable. The class verification sheets made it possible to check 72 classes quickly and gave a simple way to discuss dataset quality without opening hundreds of files manually. Without such artifacts, label errors could remain hidden until model performance drops.

The most practical lesson was that every stage needs an independent way to be checked. The dataset script, nutrition validation endpoint, macro reference tables, and frontend workflow checks were not extra polish; they were necessary for finding where a wrong output came from. This made the final report more evidence-based and made the system easier to explain.

Chapter 9

CONCLUSION

This project developed a full-stack food recognition and nutrition-assistance system for Indian meals. The system brings together a YOLO-based detector, a curated 72-class dataset, a SQLite nutrition database, a FastAPI backend, and a Next.js frontend. In the implemented workflow, a user uploads a food image, views detected foods with bounding boxes, checks macro values, adjusts serving assumptions, and receives dietary suggestions based on personal goals and health conditions.

The main objectives were achieved. Five YOLO-format source datasets were merged into a canonical Indian food dataset. The build process normalized raw labels, exported balanced splits, and produced machine-readable artifacts. Visual class verification confirmed that all 72 class names have corresponding image samples and that the class list is aligned. This was important because the dataset had to support model training, nutrition mapping, and frontend display. The nutrition pipeline produced 1,010 food records, including five verified supplemental rows, and created 103 total YOLO-to-database mappings. For the expanded dataset, all 72 classes now have nutrition mappings.

The final YOLO11s detector was trained and evaluated on the expanded dataset. The best exported model achieved 0.830 mAP@50 and 0.692 mAP@50:95 on validation, and 0.820 mAP@50 and 0.680 mAP@50:95 on the held-out test split. The backend and frontend were implemented as a working local prototype, not only as a model experiment. The backend includes image analysis, nutrition validation, macro calculation, goal, health-condition, and recommendation endpoints. The frontend handles image upload, detection visualization, macro display, serving adjustment, and recommendation review.

The work shows that food recognition becomes more useful when it is connected to nutrition databases and user-aware recommendation logic. Dataset normalization and nutrition mapping are not minor details in Indian food systems; they decide whether the

output can be trusted. The prototype still has limitations in portion estimation, annotation quality, class confusion among similar dishes, and nutrition source consistency. Even with these limitations, it gives a working foundation for a controlled recommendation model, improved portion handling, mobile use, long-term tracking, and stronger safety review.

Chapter 10

FUTURE WORK

10.1 Self-Trained Recommendation Model

The current prototype uses a language model to generate natural-language dietary suggestions from detected foods, macro values, goals, and health conditions. In a future version, this dependency should be replaced with a self-trained recommendation model. The model can be trained on curated Indian meal patterns, nutrition rules, health-condition constraints, and expert-reviewed recommendation examples. This would make the recommendation layer more controllable, easier to validate, and less dependent on external LLM services.

The self-trained model should receive structured inputs from the existing backend, including detected food classes, calories, protein, carbohydrates, fat, serving estimates, user goal, calorie target, and health profile. Its output should be constrained to safe dietary suggestions and should include confidence or rule-trace information so that users can understand why a recommendation was produced. It would also make testing easier because the same input profile should produce consistent and reviewable advice.

10.2 Portion Size Estimation

The current system reports nutrition per 100 grams and allows serving adjustment. Future versions should estimate portion size using reference objects, depth estimation, segmentation masks, or plate-size assumptions. Instance segmentation could improve food-region estimation compared with bounding boxes, especially for rice, chutneys, and mixed curries. This would improve calorie usefulness because serving size is one of the largest sources of uncertainty in image-based food logging.

10.3 Mobile Deployment

A mobile version would make the system more practical for real meal logging. The front-end could be adapted as a progressive web app or native mobile application. Depending on device constraints, the model could run through server-side inference, quantized local inference, or a hybrid approach. Camera access, offline behaviour, and response time would need careful testing because users are more likely to log food from a phone than from a desktop system.

10.4 Personalized Long-Term Tracking

Future versions can add user accounts, meal history, daily calorie budgets, trend charts, and weekly nutrition summaries. This would move the prototype from a single-image analyzer toward a long-term dietary assistant. The design should still allow users to edit detected meals manually, because long-term tracking becomes useful only when users can correct mistakes and maintain accurate records.

10.5 Clinical Safety and Explainability

Before deployment for health-sensitive users, the system should include stronger disclaimers, uncertainty indicators, and rule-based safeguards. Recommendations for conditions such as diabetes, kidney disease, hypertension, and heart disease should be reviewed by qualified nutrition professionals. The system should clearly separate general diet guidance from medical advice, especially when confidence is low or when the detected meal contains approximate nutrition mappings.

REFERENCES

- [1] R. Agarwal, T. Choudhury, N. J. Ahuja, and T. Sarkar, “IndianFoodNet: Detecting indian food items using deep learning,” *International Journal of Computational Methods and Experimental Measurements*, vol. 11, no. 4, pp. 221–232, 2023.
- [2] O. Prabhu, “Khana: A comprehensive Indian cuisine dataset,” *arXiv preprint arXiv:2509.06006*, 2025.
- [3] “Enhanced food image recognition using transfer learning,” 2024, supplied literature review paper.
- [4] K. Garisa, R. K. Kumar, and P. Singh, “Single-item training for multi-dish recognition: A class-agnostic framework for Indian food platters,” *Frontiers in Computer Science*, vol. 8, p. 1753764, 2026.
- [5] R. U. Haque, R. H. Khan, A. S. M. Shihavuddin, M. M. M. Syeed, and M. F. Uddin, “Lightweight and parameter-optimized real-time food calorie estimation from images using CNN-based approach,” *Applied Sciences*, vol. 12, no. 19, p. 9733, 2022.
- [6] Y. Arora, A. Arun, and C. V. Jawahar, “What is there in an Indian Thali?” in *Proceedings of the 16th Indian Conference on Computer Vision, Graphics and Image Processing*, 2025.
- [7] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, “Development of an Indian food composition database,” *Current Developments in Nutrition*, vol. 8, no. 7, p. 103790, 2024.
- [8] National Institute of Nutrition, “Indian Food Composition Tables,” 2017, supplied literature review reference.
- [9] “Synergistic frameworks for Indian food computing: An analysis of nutritional and visual dataset compatibility,” 2025.

- [10] M. Al-Saffar and W. R. Baiee, “Nutrition information estimation from food photos using machine learning based on multiple datasets,” *Bulletin of Electrical Engineering and Informatics*, vol. 11, no. 5, pp. 2922–2929, 2022.
- [11] A. Jha, T. Gadekar, and S. Joshi, “Label AI: Barcode scanning based mapping of nutritional values to fitness planning,” *International Journal of Computer Applications*, vol. 187, no. 54, pp. 60–68, 2025.
- [12] A. M. Saad, M. R. H. Rahi, M. M. Islam, and G. Rabbani, “Diet engine: A real-time food nutrition assistant system for personalized dietary guidance,” *Food Chemistry Advances*, vol. 7, p. 100978, 2025.
- [13] S. Hussain, N. Khan, S. A. A. Shah, S. Bano, and A. W. Khan, “AI-driven personalized meal planning: A web-based platform for tailored nutrition and health management,” *International Journal of Computer Applications*, vol. 187, no. 57, pp. 17–29, 2025.
- [14] S. Khamesian, A. Arefeen, S. M. Carpenter, and H. Ghasemzadeh, “NutriGen: Personalized meal plan generator leveraging large language models to enhance dietary and nutritional adherence,” *arXiv preprint arXiv:2502.20601*, 2025.
- [15] M. Tokic, C. Livada, T. Galba, and A. Baumgartner, “Leveraging LLMs and computer vision for personalized nutrition advice,” in *Engineering Proceedings*, vol. 125, no. 1, 2026, p. 21.
- [16] “AI-based nutrition recommendation system,” 2025, supplied literature review paper.
- [17] C. O’Hara, G. Kent, A. C. Flynn, E. R. Gibney, and C. M. Timon, “An evaluation of ChatGPT for nutrient content estimation from meal photographs,” *Nutrients*, vol. 17, no. 4, p. 607, 2025.
- [18] “Visual nutrition analysis using food segmentation and nutrient regression,” 2024, supplied literature review paper.
- [19] Roboflow Universe Contributor, “Indian food detection dataset, version 1,” <https://universe.roboflow.com/project-z0tql/indian-food-detection-kzw9g/dataset/1>, 2023, roboflow Universe export; YOLOv11 format; CC BY 4.0; local metadata exported on 28 December 2024.

- [20] —, “indian food dataset, version 6,” <https://universe.roboflow.com/microplastics-vypxl/indian-food-txofy/dataset/6>, 2025, roboflow Universe export; YOLOv11 format; CC BY 4.0; local metadata exported on 1 December 2025.
- [21] R. Agarwal, N. Bansal, T. Sarkar, T. Choudhury, and N. J. Ahuja, “IndianFood-7 dataset, version 2,” https://universe.roboflow.com/indianfood/indian_food-pwzlc/dataset/2, 2024, roboflow Universe export; YOLOv11 format; CC BY 4.0; local metadata exported on 24 October 2024.
- [22] —, “IndianFoodNet-30 dataset, version 1,” <https://universe.roboflow.com/indianfoodnet/indianfoodnet/dataset/1>, 2023, roboflow Universe export; YOLOv8 format; CC BY 4.0; local metadata exported on 20 April 2023.
- [23] Roboflow Universe Contributor, “south indian food detection dataset, version 19,” <https://universe.roboflow.com/bharani-lucid/south-indian-food-detection/dataset/19>, 2024, roboflow Universe export; YOLOv11 format; CC BY 4.0; local metadata exported on 24 October 2024.
- [24] FastAPI Project, “FastAPI documentation,” <https://fastapi.tiangolo.com/>, 2024, software documentation. Accessed: 6 June 2026.
- [25] Vercel, “Next.js documentation,” <https://nextjs.org/docs>, 2024, software documentation. Accessed: 6 June 2026.
- [26] “Evolve bhakarwadi nutrition facts per 100 g,” <https://www.fatsecret.co.in/calories-nutrition/evolve/bhakarwadi/100g>, 2026, FatSecret India. Accessed: 6 June 2026.
- [27] “Ghevar recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/ghevar/>, 2026, Clearcals. Accessed: 6 June 2026.
- [28] “Jalebi recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/jalebi/>, 2026, Clearcals. Accessed: 6 June 2026.
- [29] “Khandvi recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/khandvi/>, 2026, Clearcals. Accessed: 6 June 2026.
- [30] “Nandu Kari recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/nandu-kari/>, 2026, Clearcals. Accessed: 6 June 2026.

Appendix A

DATASET CLASSES AND NUTRITION MAPPINGS

A.1 Expanded Dataset Class List

The final dataset contains 72 canonical classes. The class list is stored in the generated dataset configuration file, and visual verification confirmed that every class has image samples. The mapping table and per-class count table below include the complete class set so that the training dataset can be checked directly from the report.

A.2 Class-to-INDB Mapping Table

Table A.1 lists the database food name used for each dataset class. Most rows come from INDB. The few classes absent from the available INDB sheet use verified supplemental rows, as noted in the project database. The table is included here so that the nutrition assumptions are visible instead of remaining only inside the generated SQLite file.

Table A.1: Full 72-class dataset to INDB mapping table

Dataset class	Matched database food name	Score
aloo_gobi	Potato cauliflower (Aloo gobhi)	1.00
aloo_masala	Potato curry (Aloo ki sabzi)	0.90
appam	Appam	1.00
beetroot_poriyal	Vegetables stir fry	0.75

Dataset class	Matched database food name	Score
besan_cheela	Gram flour chilla/cheela (Besan chilla/cheela)	1.00
bhakarwadi	Bhakarwadi	1.00
bhakri	Chapati/Roti	0.80
bhatura	Bhatura	1.00
bhindi_masala	Okra/Lady's fingers fry (Bhindi sabzi/sabji/subji)	0.95
biryani	Vegetable biryani/biriyani	0.85
carrot_poriyal	Carrot and cabbage with coconut (Nariyal ke saath pattagobhi aur gajar)	0.85
chai	Hot tea (Garam Chai)	1.00
chicken	Chicken curry	0.85
chicken_65	Chilli chicken	0.80
chicken_biryani	Chicken pulao	0.80
chole	Chickpeas curry (Safed channa curry)	1.00
coconut_chutney	Coconut chutney (Nariyal ki chutney)	1.00
dal	Mixed dal	0.90
dhokla	Dhokla	1.00
dosa	Plain dosa	0.95
dum_aloo	Dum aloo	1.00
eggs	Boiled egg (Ubla anda)	0.95
fish_curry	Fish curry (Machli curry)	1.00
ghevar	Ghevar	1.00
green_chutney	Green chutney	1.00
gulab_jamun	Gulab Jamun with khoya	1.00
idli	Idli	1.00
jalebi	Jalebi	1.00
kaara_chutney	Tomato chutney (Tamatar ki chutney)	0.85
kali	Maize porridge	0.70

Dataset class	Matched database food name	Score
kebab	Boti kebab	0.90
khandvi	Khandvi	1.00
kheer	Rice kheer (Chawal ki kheer)	1.00
koozh	Maize porridge	0.70
kulfi	Kulfi	1.00
lassi	Sweet Lassi (Meethi lassi)	0.95
lemon_rice	Lemon rice (Pulihora, Elumichai sadam, Chitranna)	1.00
medu_vada	Plain urad dal vada (Uzunne vada/Minapa garelu/Ulundu vadai/Medu vada)	1.00
modak	Semolina laddoo with coconut (Suji/Rava aur nariyal ke laddoo)	0.70
mushroom_biryani	Mushroom pulao	0.80
mutton_biryani	Mutton biryani/biriyani	1.00
mutton_curry	Mutton korma	0.85
nandu_masala	Nandu Kari (Crab masala)	1.00
nei_satham	Plain pulao	0.80
omelette	Plain omelette/omlet	1.00
onion_pakoda	Onion pakora/pakoda (Pyaz ke pakode)	1.00
paal_kolukattai	Rice kheer (Chawal ki kheer)	0.75
palak_paneer	Spinach paneer (Palak paneer)	1.00
paneer_biryani	Paneer pulao	0.80
paratha	Plain parantha/paratha	0.95
parupu_vadai	Fermented bengal gram vada (Khameerikrit/Ufna hua channa dal ka vada)	0.90
pidi_kolukattai	Rice puttlu (Ari puttlu)	0.75
poha	Poha	1.00
poorna_kolukattai	Semolina laddoo with coconut (Suji/Rava aur nariyal ke laddoo)	0.70
prawn_thokku	Prawn curry (with coconut) (Jhinga curry)	0.85

Dataset class	Matched database food name	Score
puri	Poori	1.00
raita	Cucumber raita (Kheere ka raita)	0.90
rajma_curry	Kidney bean curry (Rajmah curry)	1.00
ras_malai	Rasmalai	1.00
rice	Boiled rice (Uble chawal)	1.00
roti	Chapati/Roti	1.00
saag	Sarson ka saag	1.00
salad	Tossed salad	0.90
sambar	Sambar	1.00
sambar_satham	Vegetable khichdi/khichri	0.80
samosa	Potato samosa (Aloo ka samosa)	1.00
shahi_paneer	Shahi paneer	1.00
thepla	Methi thepla	0.95
upma	Semolina upma (Suji/Rava upma)	0.95
veg_briyani	Vegetable biryani/biriyani	1.00
veg_pulao	Mixed vegetable pulao	1.00
ven_pongal	Plain khitchdi (Plain khichri/khichdi)	0.75

A.3 Known Annotation Issues

Table A.2: Sample-level annotation issues found during visual verification

File	Finding
train27229-jalebi.jpg	Labeled as jalebi, but the image does not visually look like jalebi.
train18550-kaara-chutney label	kaara_chutney box is placed on the vada instead of the visible chutney.
train32747-nandu-masala label	Full image is crab, but the bounding box is extremely thin and produces a blank crop.

A.4 Visual Verification Sheet Set

The following figures reproduce the complete visual verification sheet set. The original eight contact sheets were divided into top and bottom halves, producing sixteen larger half-sheets for clearer reading in the printed report. The sheets were generated from actual dataset files, with sampled full images and label-based crops used to check whether class names matched the visible food items. These sheets helped during review because they gave a quick visual check of the class list without opening the dataset folder manually.

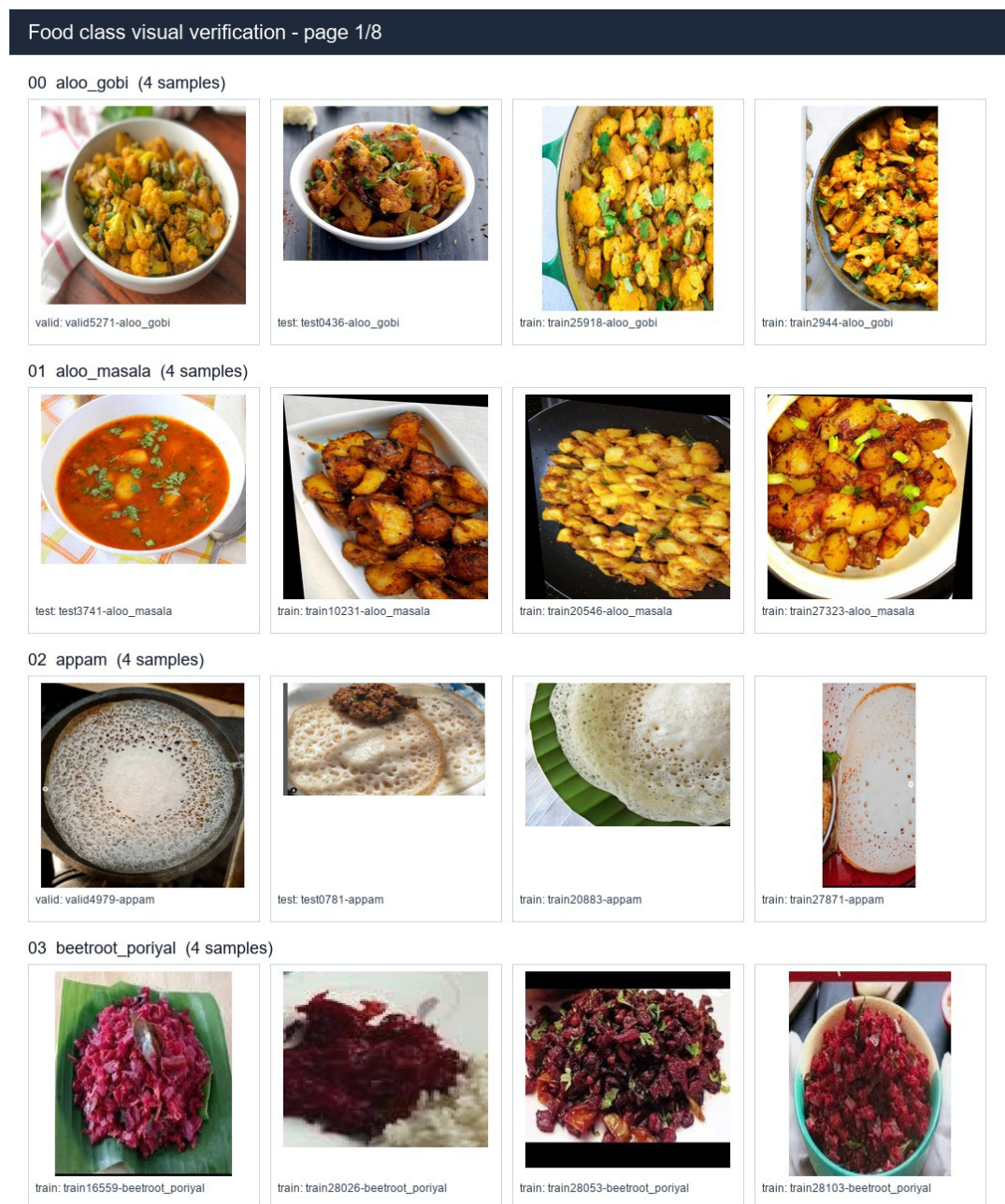


Figure A.1: Visual verification half-sheet 1

04 besan_cheela (4 samples)



train: train19866-besan_cheela



train: train28199-besan_cheela



train: train8435-besan_cheela



train: train9492-besan_cheela

05 bhakarwadi (4 samples)



train: train17722-bhakarwadi



train: train27451-bhakarwadi



train: train28433-bhakarwadi



train: train28566-bhakarwadi

06 bhakri (4 samples)



train: train22655-bhakri



train: train27259-bhakri



train: train28610-bhakri



train: train28619-bhakri

07 bhatura (4 samples)



train: train15921-bhatura



train: train18107-bhatura



train: train21814-bhatura



train: train8316-bhatura

08 bhindi_masala (4 samples)



train: train15383-bhindi_masala



train: train16647-bhindi_masala



train: train3302-bhindi_masala



train: train6328-bhindi_masala

Figure A.2: Visual verification half-sheet 2

Food class visual verification - page 2/8

09 biryani (4 samples)



test: test4454-biryani



train: train32656-biryani



train: train7230-biryani



train: train9359-biryani

10 carrot_poriyal (4 samples)



test: test3061-carrot_poriyal



test: test5420-carrot_poriyal



train: train0298-carrot_poriyal



train: train28850-carrot_poriyal

11 chai (4 samples)



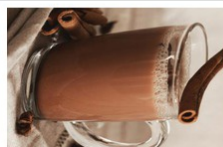
valid: valid1186-chai



train: train0131-chai



train: train10963-chai



train: train6235-chai

12 chicken (4 samples)



test: test3819-chicken



train: train11883-chicken



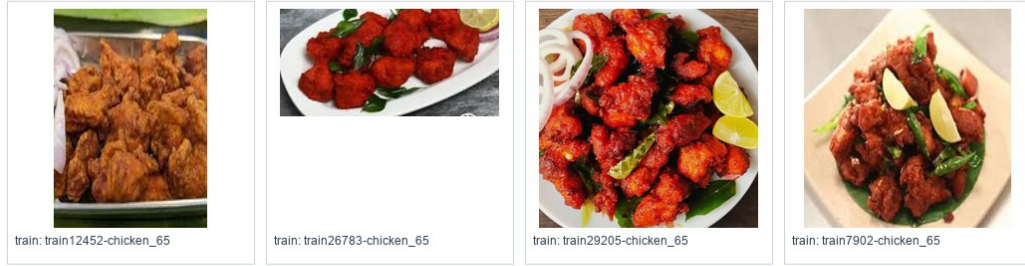
train: train21858-chicken



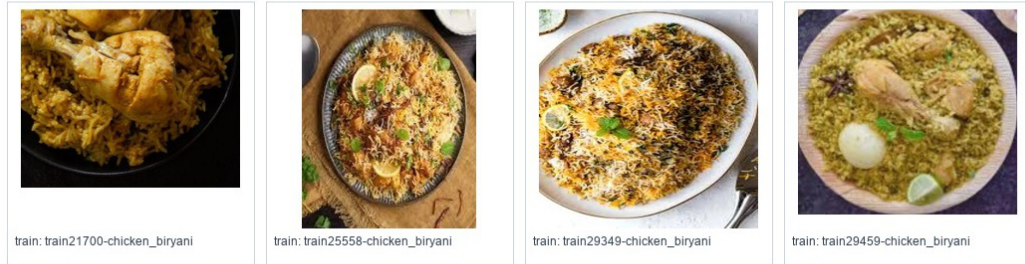
train: train23068-chicken

Figure A.3: Visual verification half-sheet 3

13 chicken_65 (4 samples)



14 chicken_biryani (4 samples)



15 chole (4 samples)



16 coconut_chutney (4 samples)



17 dal (4 samples)

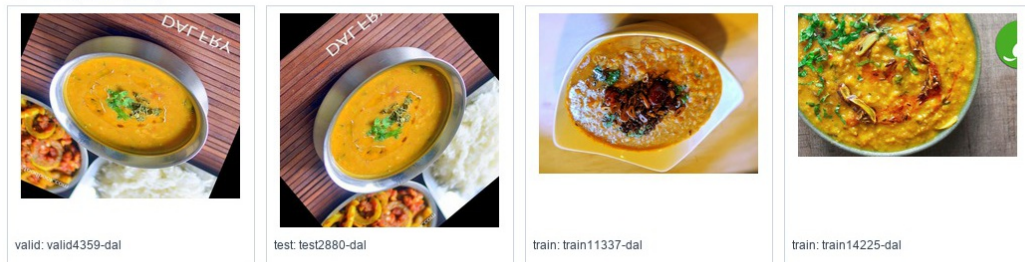


Figure A.4: Visual verification half-sheet 4

Food class visual verification - page 3/8

18 dhokla (4 samples)



valid: valid3146-dhokla



valid: valid4860-dhokla



train: train22854-dhokla



train: train7879-dhokla

19 dosa (4 samples)



valid: valid3782-dosa



train: train13398-dosa



train: train14157-dosa



train: train3629-dosa

20 dum_aloo (4 samples)



valid: valid0393-dum_aloo



test: test1910-dum_aloo



test: test4081-dum_aloo



train: train15102-dum_aloo

21 eggs (4 samples)



valid: valid0879-eggs



train: train18161-eggs



train: train18289-eggs



train: train22967-eggs

Figure A.5: Visual verification half-sheet 5

22 fish_curry (4 samples)



train: train19842-fish_curry



train: train21203-fish_curry



train: train26873-fish_curry



train: train8741-fish_curry

23 ghevar (4 samples)



train: train17688-ghevar



train: train23125-ghevar



train: train29903-ghevar

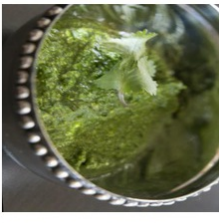


train: train3832-ghevar

24 green_chutney (4 samples)



test: test4663-green_chutney



train: train11511-green_chutney



train: train22272-green_chutney



train: train9701-green_chutney

25 gulab_jamun (4 samples)



train: train29982-gulab_jamun



train: train30012-gulab_jamun



train: train30038-gulab_jamun



train: train5779-gulab_jamun

26 idli (4 samples)



train: train0002-idli



train: train10642-idli



train: train20767-idli



train: train2333-idli

Figure A.6: Visual verification half-sheet 6

Food class visual verification - page 4/8

27 jalebi (4 samples)



valid: valid5715-jalebi



test: test1540-jalebi



train: train27229-jalebi



train: train4927-jalebi

28 kaara_chutney (4 samples)



train: train18550-kaara_chutney



train: train30257-kaara_chutney



train: train30323-kaara_chutney



train: train3734-kaara_chutney

29 kali (4 samples)



valid: valid3458-kali



test: test2142-kali



train: train30464-kali



train: train30509-kali

30 kebab (4 samples)



train: train2634-kebab



train: train30687-kebab



train: train30764-kebab



train: train30774-kebab

Figure A.7: Visual verification half-sheet 7

31 khandvi (4 samples)



valid: valid1049-khandvi



train: train10061-khandvi



train: train3645-khandvi



train: train4209-khandvi

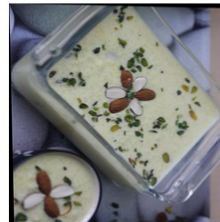
32 kheer (4 samples)



train: train11078-kheer



train: train31057-kheer



train: train3978-kheer



train: train8782-kheer

33 koozh (4 samples)



train: train20855-koozh



train: train24935-koozh

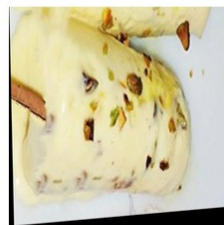


train: train27478-koozh



train: train31296-koozh

34 kulfi (4 samples)



test: test1333-kulfi



train: train19142-kulfi



train: train2846-kulfi



train: train31442-kulfi

35 lassi (4 samples)



train: train2554-lassi



train: train31646-lassi



train: train31662-lassi

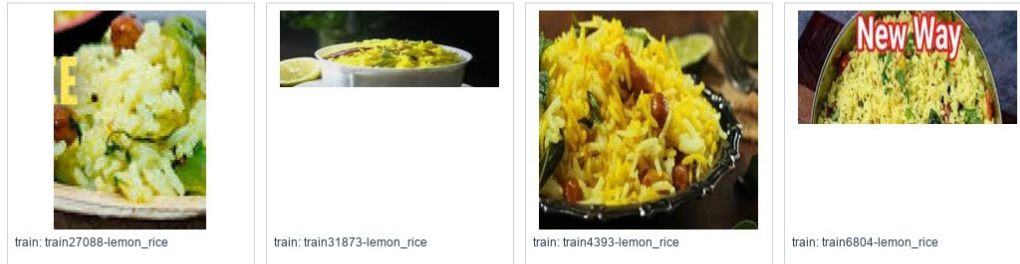


train: train6641-lassi

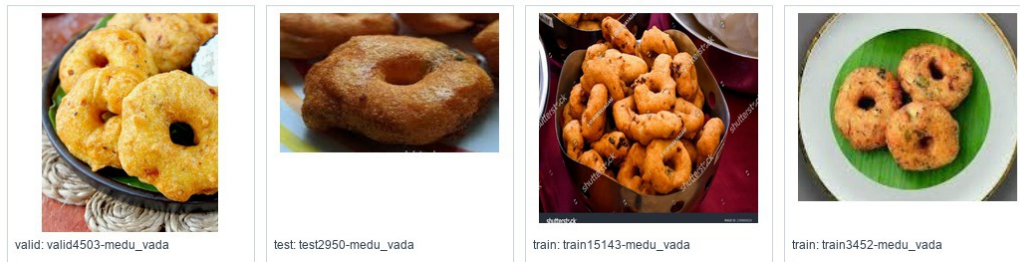
Figure A.8: Visual verification half-sheet 8

Food class visual verification - page 5/8

36 lemon_rice (4 samples)



37 medu_vada (4 samples)



38 modak (4 samples)



39 mushroom_biryani (4 samples)

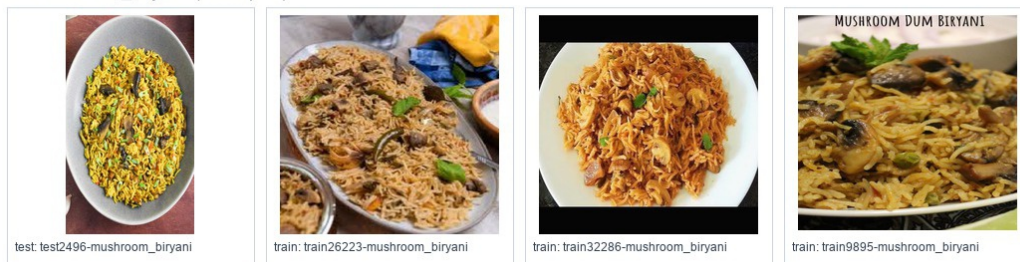
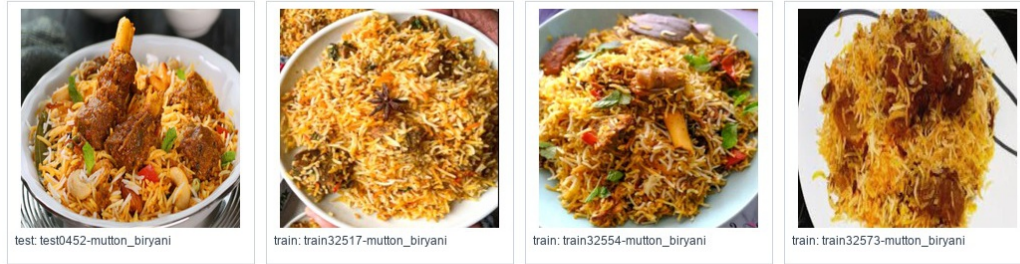


Figure A.9: Visual verification half-sheet 9

40 mutton_biryani (4 samples)



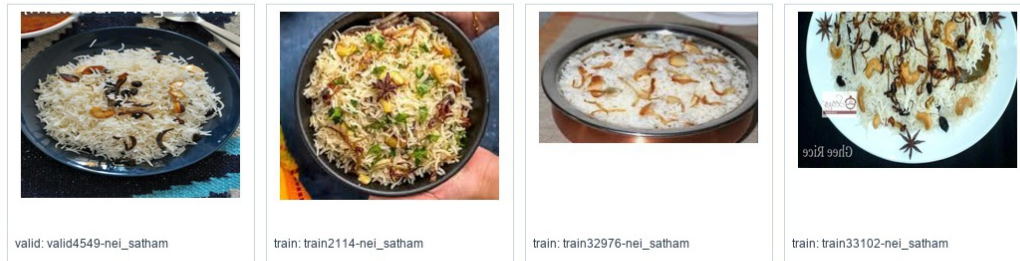
41 mutton_curry (4 samples)



42 nandu_masala (4 samples)



43 nei_satham (4 samples)



44 omelette (4 samples)

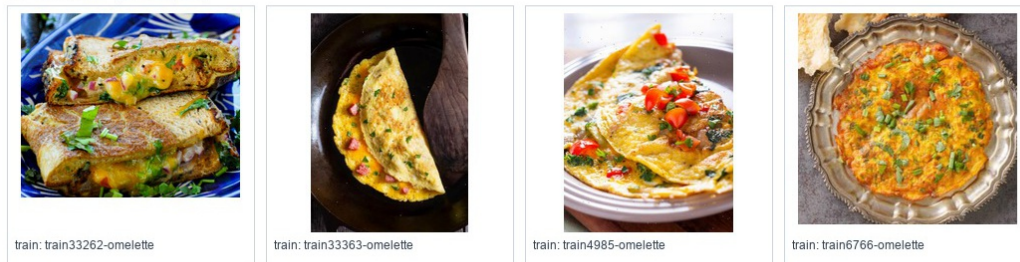


Figure A.10: Visual verification half-sheet 10

Food class visual verification - page 6/8

45 onion_pakoda (4 samples)



valid: valid0147-onion_pakoda



valid: valid4504-onion_pakoda



train: train21948-onion_pakoda



train: train7807-onion_pakoda

46 paal_kolukattai (4 samples)



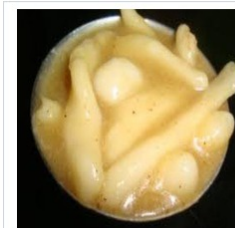
train: train18614-paal_kolukattai



train: train25793-paal_kolukattai

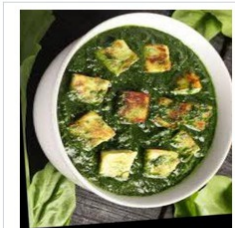


train: train33605-paal_kolukattai



train: train8818-paal_kolukattai

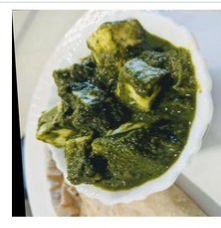
47 palak_paneer (4 samples)



valid: valid2611-palak_paneer



test: test0559-palak_paneer



train: train10779-palak_paneer



train: train19876-palak_paneer

48 paneer_biryani (4 samples)



valid: valid4561-paneer_biryani



train: train25553-paneer_biryani



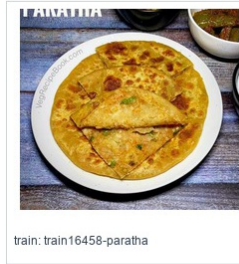
train: train26234-paneer_biryani



train: train33856-paneer_biryani

Figure A.11: Visual verification half-sheet 11

49 paratha (4 samples)



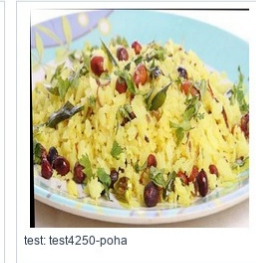
50 parupu_vadai (4 samples)



51 pidi_kolukattai (4 samples)



52 poha (4 samples)



53 poorna_kolukattai (4 samples)



Figure A.12: Visual verification half-sheet 12

Food class visual verification - page 7/8

54 prawn_thokku (4 samples)



train: train34759-prawn_thokku



train: train34798-prawn_thokku



train: train34839-prawn_thokku



train: train6231-prawn_thokku

55 puri (4 samples)



train: train34935-puri



train: train34991-puri



train: train35023-puri



train: train8919-puri

56 raita (4 samples)



valid: valid4965-raita



train: train25244-raita

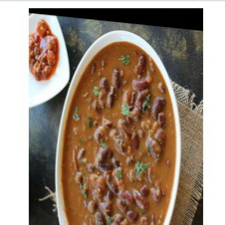


train: train35114-raita



train: train35209-raita

57 rajma_curry (4 samples)



valid: valid5469-rajma_curry



train: train0065-rajma_curry



train: train4039-rajma_curry



train: train9691-rajma_curry

Figure A.13: Visual verification half-sheet 13

58 ras_malai (4 samples)



train: train35272-ras_malai



train: train35280-ras_malai

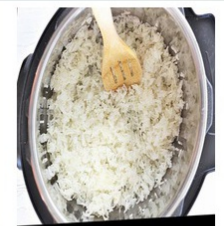


train: train35325-ras_malai

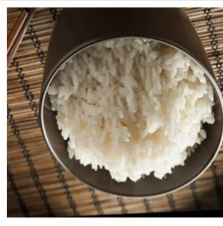


train: train35420-ras_malai

59 rice (4 samples)



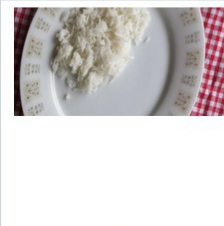
test: test4926-rice



train: train14379-rice



train: train26538-rice



train: train4234-rice

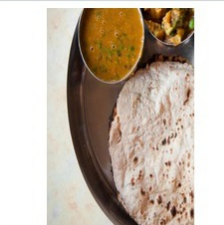
60 roti (4 samples)



valid: valid4952-roti



test: test0535-roti



train: train22320-roti



train: train9152-roti

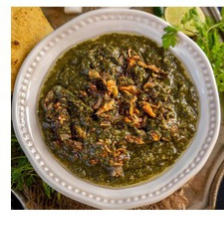
61 saag (4 samples)



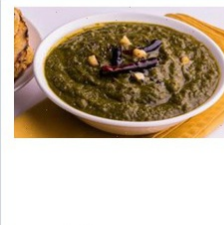
valid: valid0447-saag



test: test3110-saag



train: train25328-saag



train: train9146-saag

62 salad (4 samples)



train: train10948-salad



train: train21338-salad



train: train35491-salad



train: train35510-salad

Figure A.14: Visual verification half-sheet 14

Food class visual verification - page 8/8

63 sambar (4 samples)



64 sambar_satham (4 samples)



65 samosa (4 samples)



66 shahi_paneer (4 samples)



Figure A.15: Visual verification half-sheet 15

67 thepla (4 samples)



test: test2116-thepla



test: test2394-thepla



train: train10985-thepla



train: train36057-thepla

68 upma (4 samples)



train: train13455-upma



train: train23242-upma



train: train36151-upma



train: train36236-upma

69 veg_briyani (4 samples)



train: train20201-veg_briyani



train: train26450-veg_briyani



train: train36323-veg_briyani



train: train36422-veg_briyani

70 veg_pulao (4 samples)



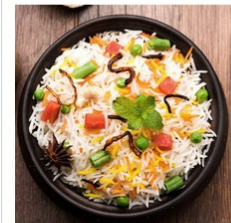
train: train10944-veg_pulao



train: train36475-veg_pulao



train: train36497-veg_pulao



train: train4475-veg_pulao

71 ven_pongal (4 samples)



valid: valid5163-ven_pongal



train: train23259-ven_pongal



train: train24352-ven_pongal



train: train36683-ven_pongal

Figure A.16: Visual verification half-sheet 16

A.5 Per-Class Dataset Counts

Table A.3: Per-class dataset counts after merge and train augmentation

Class	Train before → after	Valid	Test
aloo_gobi	556 → 568	119	119
aloo_masala	520 → 528	111	112
appam	210 → 421	45	45
beetroot_poriyal	210 → 421	45	45
besan_cheela	207 → 414	44	45
bhakarwadi	256 → 500	55	55
bhakri	79 → 168	17	17
bhatura	680 → 680	145	146
bhindi_masala	1046 → 1056	224	224
biryani	700 → 703	150	150
carrot_poriyal	210 → 421	45	45
chai	340 → 511	73	73
chicken	925 → 925	198	198
chicken_65	210 → 421	45	45
chicken_biryani	210 → 420	45	45
chole	1277 → 1362	273	274
coconut_chutney	1007 → 1207	216	216
dal	970 → 985	208	208
dhokla	703 → 704	151	151
dosa	1125 → 1227	241	241
dum_aloo	358 → 500	77	77
eggs	519 → 561	111	111
fish_curry	362 → 500	77	78
ghevar	235 → 470	50	51
green_chutney	661 → 748	141	142
gulab_jamun	272 → 501	58	59
idli	821 → 889	176	176

Class	Train before → after	Valid	Test
jalebi	872 → 872	187	187
kaara_chutney	218 → 457	46	47
kali	210 → 423	45	45
kebab	161 → 322	34	35
khandvi	325 → 500	70	70
kheer	294 → 500	63	63
koozh	210 → 421	45	45
kulfi	332 → 500	71	71
lassi	322 → 500	69	69
lemon_rice	210 → 420	45	45
medu_vada	386 → 522	83	83
modak	120 → 240	26	26
mushroom_biryani	210 → 420	45	45
mutton_biryani	210 → 421	45	45
mutton_curry	350 → 500	75	75
nandu_masala	210 → 420	45	45
nei_satham	210 → 420	45	45
omelette	212 → 440	45	46
onion_pakoda	333 → 500	71	72
paal_kolukattai	210 → 420	45	45
palak_paneer	1000 → 1000	214	214
paneer_biryani	210 → 420	45	45
paratha	127 → 282	27	28
parupu_vadai	210 → 425	45	45
pidi_kolukattai	210 → 422	45	45
poha	1049 → 1058	224	225
poorna_kolukattai	210 → 421	45	45
prawn_thokku	210 → 420	45	45
puri	380 → 535	81	82

Class	Train before → after	Valid	Test
raita	239 → 554	51	52
rajma_curry	1008 → 1050	216	216
ras_malai	322 → 500	69	69
rice	1776 → 1856	380	381
roti	803 → 821	172	172
saag	525 → 525	112	113
salad	157 → 360	34	34
sambar	475 → 617	102	102
sambar_satham	210 → 420	45	45
samosa	358 → 500	77	77
shahi_paneer	878 → 878	188	189
thepla	174 → 377	37	38
upma	145 → 290	31	31
veg_briyani	210 → 420	45	45
veg_pulao	170 → 364	36	37
ven_pongal	210 → 433	45	45

Appendix B

API RESPONSE EXAMPLES

B.1 Analyze Food Response

The following listing shows the simplified structure of a successful image analysis response. The backend response follows this structure so that the frontend can render detection boxes, nutrition cards, and macro totals from predictable fields.

Listing B.1: Simplified food analysis response

```
{
  "success": true,
  "food_detected": true,
  "image_width": 760,
  "image_height": 787,
  "detections": [
    {
      "food_label": "Chole",
      "display_name": "Chickpeas curry (Safed channa curry)",
      "confidence": 0.95,
      "bbox": {
        "x1": 442,
        "y1": 194,
        "x2": 697,
        "y2": 551
      },
      "macros": {
        "calories": 163.4,
        "protein_g": 6.1,
        "carbs_g": 20.0,
```

```
        "fat_g": 6.8
      },
      "nutrition_source": "INDB"
    }
  ]
}
```

B.2 Macro Calculation Request

Listing B.2: Macro calculation request

```
{
  "goal": "weight_loss",
  "target_calories": 2000,
  "health_condition": "diabetic"
}
```

B.3 Macro Calculation Response

Listing B.3: Macro calculation response

```
{
  "goal": "weight_loss",
  "target_calories": 2000,
  "health_condition": "diabetic",
  "macros": {
    "carbs_percent": 33,
    "protein_percent": 41,
    "fat_percent": 26,
    "carbs_g": 165,
    "protein_g": 205,
    "fat_g": 58
  }
}
```

B.4 No Food Response

Listing B.4: No-food response shape

```
{
  "success": true,
  "food_detected": false,
  "food_not_found": true,
  "detections": [],
  "message": "No food items detected in the uploaded image."
}
```

B.5 Macro Target Reference Tables

The following tables show the macro targets generated for a 2,000 kcal daily target. The backend normalizes goal and health-condition modifiers before converting calories to grams. These tables are included as reference outputs so that the macro calculation logic can be checked without running the application.

Table B.1: Weight-loss macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	40	35	25	200	175	56
Diabetic	33	41	26	165	205	58
Hypertension	41	36	23	205	180	51
Heart disease	43	36	21	215	180	47
High cholesterol	42	39	19	210	195	42
Digestive issues	43	34	23	215	170	51
Kidney disease	50	23	27	250	115	60
Anemia	40	38	22	200	190	49
Thyroid disorder	39	36	25	195	180	56

Table B.2: Muscle-gain macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	40	30	30	200	150	67
Diabetic	33	36	31	165	180	69
Hypertension	41	31	28	205	155	62
Heart disease	44	31	25	220	155	56
High cholesterol	43	34	23	215	170	51
Digestive issues	43	29	28	215	145	62
Kidney disease	49	19	32	245	95	71
Anemia	40	33	27	200	165	60
Thyroid disorder	39	31	30	195	155	67

Table B.3: Maintenance macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	50	25	25	250	125	56
Diabetic	43	31	26	215	155	58
Hypertension	51	26	23	255	130	51
Heart disease	54	26	20	270	130	44
High cholesterol	53	28	19	265	140	42
Digestive issues	53	24	23	265	120	51
Kidney disease	59	15	26	295	75	58
Anemia	50	28	22	250	140	49
Thyroid disorder	49	26	25	245	130	56

Table B.4: Endurance macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	55	20	20	275	100	44
Diabetic	51	26	23	255	130	51
Hypertension	59	22	19	295	110	42
Heart disease	62	21	17	310	105	38
High cholesterol	61	23	16	305	115	36
Digestive issues	61	20	19	305	100	42
Kidney disease	66	13	21	330	65	47
Anemia	58	23	19	290	115	42
Thyroid disorder	57	22	21	285	110	47